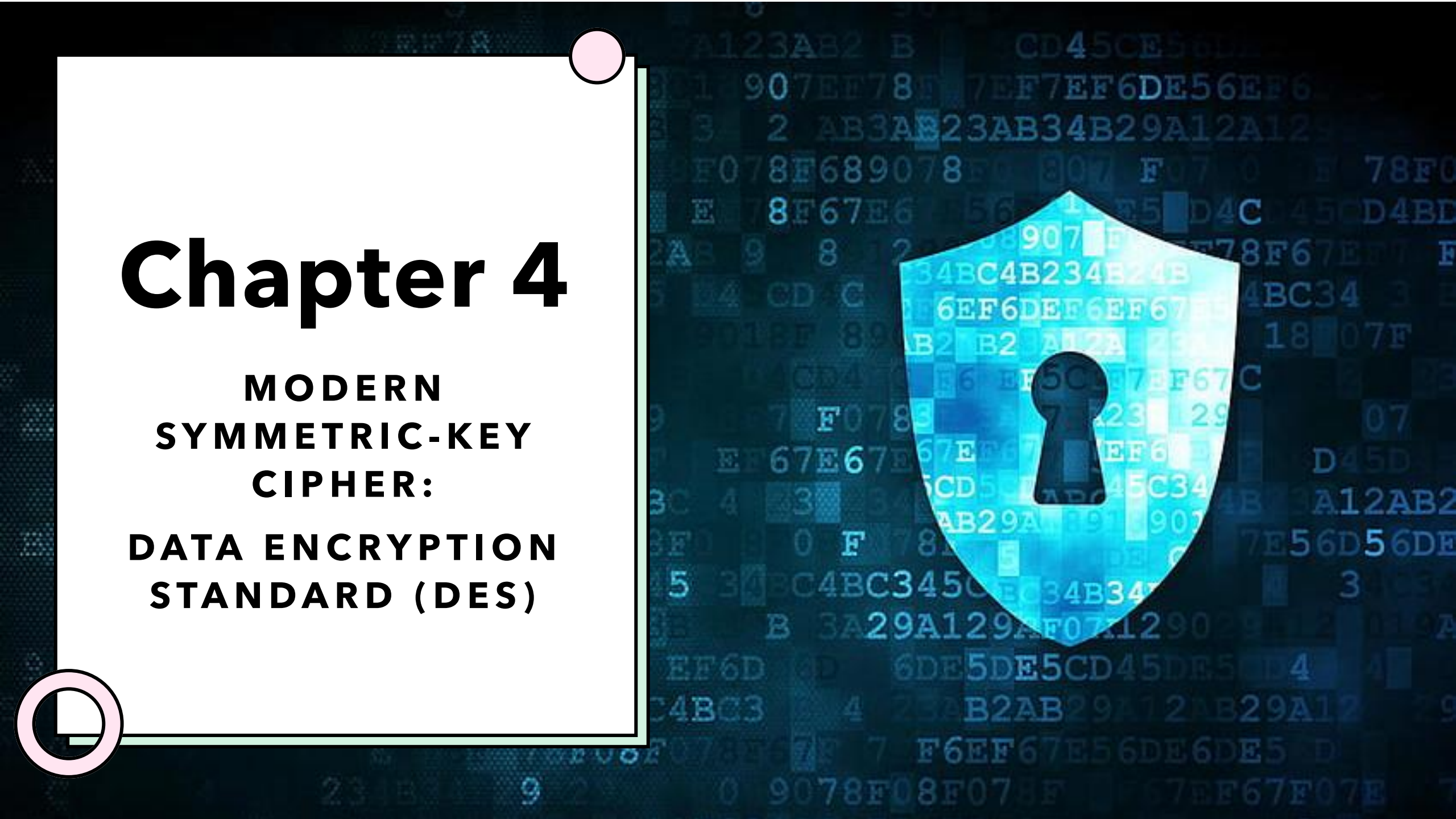
# Chapter 4

## MODERN SYMMETRIC-KEY CIPHER:

## DATA ENCRYPTION STANDARD (DES)



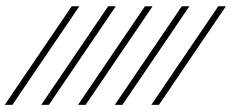
# Outlines

1. Types of modern symmetric ciphers
2. Feistel cipher
3. Data Encryption Standard (DES)
4. Variants of DES

# Learning Objectives

**In the end of this Chapter 4, students will be able:**

1. To understand the **operations and differences** between the two major symmetric ciphers of **block cipher** and **stream cipher**.
2. To understand the structure of the **Feistel cipher**.
3. To learn and **perform the computations** surrounding the **Data Encryption Standard (DES)**.
4. To learn and understand some **variants of DES**.



# STREAM CIPHERS
# &
# BLOCK CIPHERS

# Types of Symmetric Cipher - Overview

❖ Symmetric ciphers can generally be sub-divided into two major types:

1. **Stream Cipher**
   - ➢ The data that is operated (encrypted) as a **stream** with one bit or one byte at a time.
   - ➢ Examples: Vigenère cipher and Vernam cipher.
2. **Block Cipher**
   - ➢ The data is split into **smaller blocks of fixed size**, and the operation is performed block-by-block to produce ciphertext block of equal size (length).
   - ➢ Example: Keyed Transposition cipher.

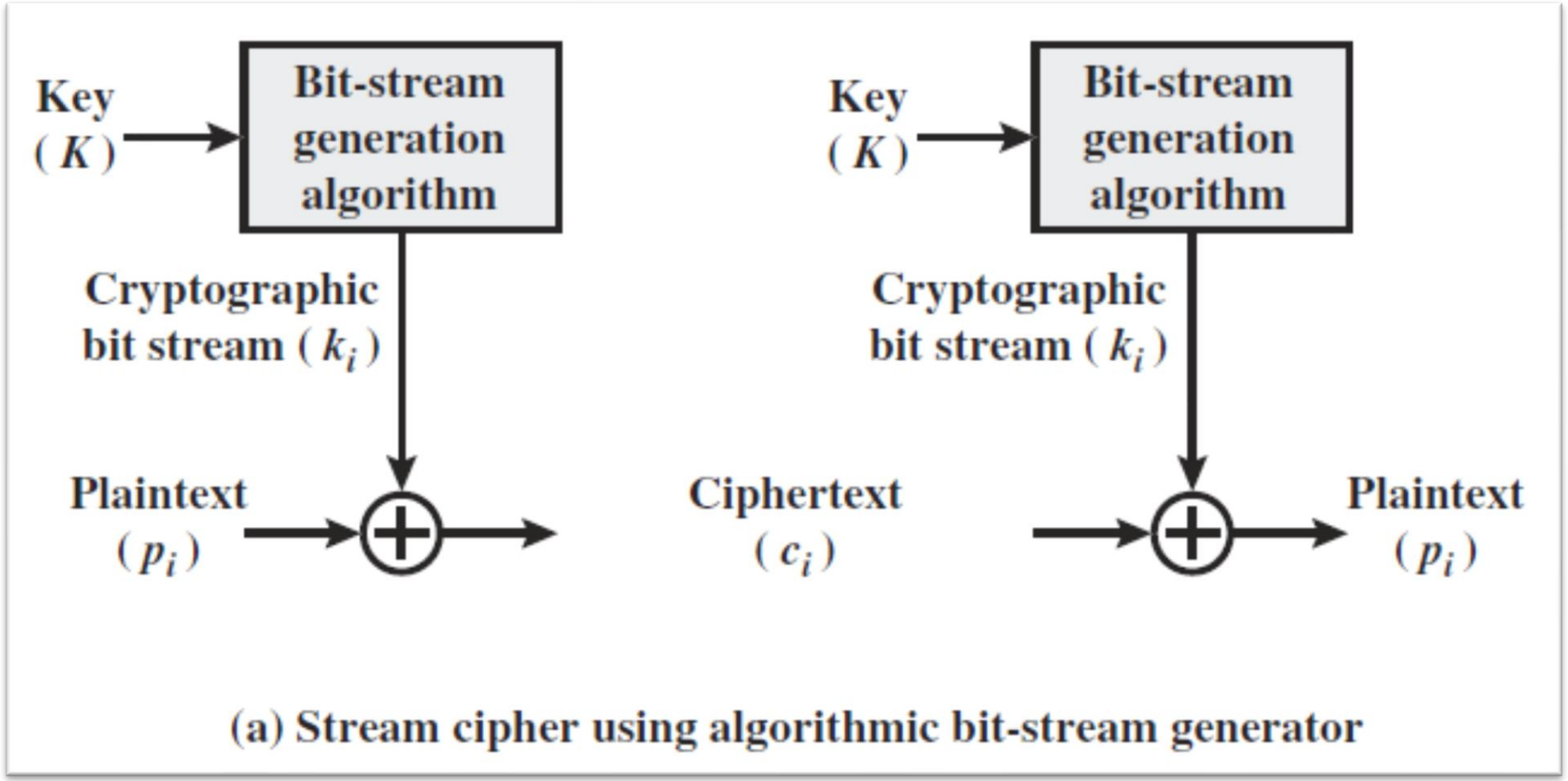
# Stream Ciphers

- The data that is operated (encrypted) as a **stream** with **one bit** or **one byte** at a time.
- The plaintext bits are encrypted with a **pseudorandom bit-stream** (**keystream**), typically by an **XOR (exclusive-OR) operation**.
- **Examples** of modern stream ciphers:
  1. Rivest Cipher 4 (RC4)
  2. Software Optimized Encryption Algorithm (SEAL)
  3. Very Efficient Substitution Transposition (VEST)

# Stream Ciphers

- **Advantage**:
  - It executes at a higher speed than block cipher and have low hardware complexity.
  - If the cryptographic **keystream is random**, then this cipher is **unbreakable** by any means other than acquiring the keystream.
- **Disadvantage**:
  - The distribution of the random keystream is difficult in a larger data traffic (**key distribution problem**).
    - **Solution**: Instead of distributing the long keystream, a **bit (keystream) generator** is distributed, so that users can generate the keystream whenever necessary.

# Stream Ciphers




(a) Stream cipher using algorithmic bit-stream generator

# Block Ciphers

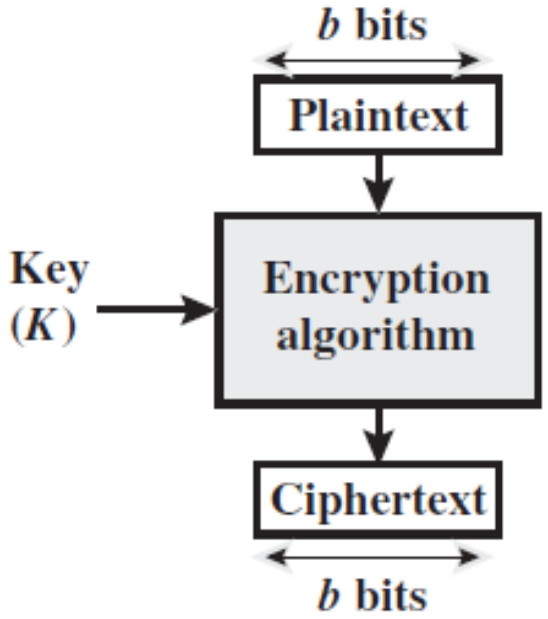
- The data is split into **smaller blocks** of **fixed size**, and the operation is performed block-by-block to produce ciphertext block of equal size (length).
- Typically, a block size of 64,128 or **256 (current deployment standard)** bits is used.
- Before processing a block cipher, all the bits in the block should be available. If a block does not achieve the specific required size, **padding** is performed.
- Since operation is done on a block, the bits in a block can be viewed as a stream.

# Block Ciphers

❖ **Examples** of block ciphers:

1. Data Encryption Standard (DES)
2. Double DES,
3. Triple DES,
4. Advanced Encryption Standard (AES),
5. Rivest Cipher 5 (RC5),
6. Blowfish

- **Note:** We will learn some modes of operation in Chapter 6 later and observe that block cipher can be used to achieve the same effect as stream cipher.




(b) Block cipher

# FEISTEL CIPHER STRUCTURES (MOTIVATION)

# 1st Motivation – **Block Cipher**

- A block cipher operates on a **plaintext block of $n$ bits** to produce a **ciphertext block of $n$ bits**. Hence there are **$2^n$ possible different plaintext blocks** involved and **$2^n$ possible ciphertext blocks produced**.
- For encryption and decryption to be reversible, the **map must be unique** (**one-to-one correspondence**).

**Reversible Mapping**

| Plaintext | Ciphertext |
|---|---|
| 00 | 11 |
| 01 | 10 |
| 10 | 00 |
| 11 | 01 |

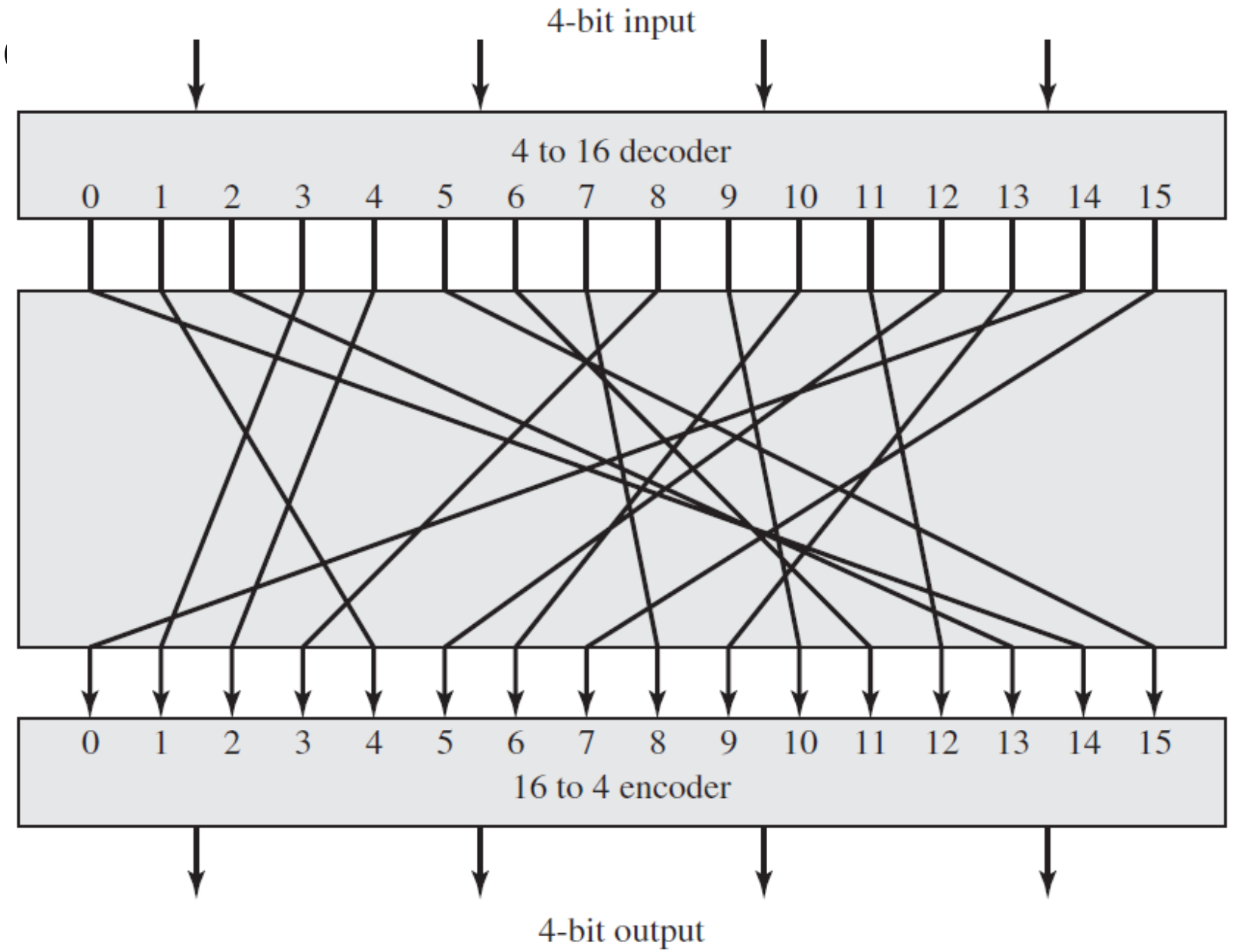
**Irreversible Mapping**

| Plaintext | Ciphertext |
|---|---|
| 00 | 11 |
| 01 | 10 |
| 10 | 01 |
| 11 | 01 |

The map is **not unique**, as two plaintexts have the same ciphertext.

# 2nd Motivation – **Product Cipher**

- In the end of Chapter 3, we learn that **product cipher** combined several technique such as substitution and permutation.
- The concept of product cipher was introduced by **Claude Shannon** in 1949.
- **Idea**: To enable the block ciphers to achieve two important properties:
  - **Diffusion**: To hide the relationships between the **plaintext and ciphertext**. This is done through **permutation**.
  - **Confusion**: To hide the relationships between the **key and ciphertext**. This is achieved through **substitution**.
- In next slide, an example of how '**confusion**' is achieved in substitution, and it is called ***IDEAL*** **block cipher**.




| Plaintext | Ciphertext |
|---|---|
| 0000 | 1110 |
| 0001 | 0100 |
| 0010 | 1101 |
| 0011 | 0001 |
| 0100 | 0010 |
| 0101 | 1111 |
| 0110 | 1011 |
| 0111 | 1000 |
| 1000 | 0011 |
| 1001 | 1010 |
| 1010 | 0110 |
| 1011 | 1100 |
| 1100 | 0101 |
| 1101 | 1001 |
| 1110 | 0000 |
| 1111 | 0111 |

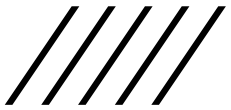
# 3rd Motivation – **Practical Cipher**

❖Although ***IDEAL*** block cipher is ideal, this cipher has some practical issues:

1. If a **small block size** (such as $n = 4$) is used, the system is equivalent to a classical substitution cipher, and it is **vulnerable to a statistical analysis. 16 combinations of 4 bits. =** $\mathbf{2 \times 2^4 = 2^5}$ $bits$
2. If a **block size is large**, then statistical analysis is infeasible, but this cipher is **not practical** from an implementation and performance point of view.

❖ For an $n$ -bit general substitution block cipher, the size of the key is $n \times 2^n$.

- **Example**: A 64-bit block, which is a desirable length to thwart statistical attacks, has the key size of $64 \times 2^{64} = 2^{70} \approx 10^{21}$ bits. This is certainly **not practical** to be used.
- S-box = Substitution boxes



# FEISTEL CIPHER STRUCTURES

# **Feistel Cipher** Structure – **Optimal Solution**

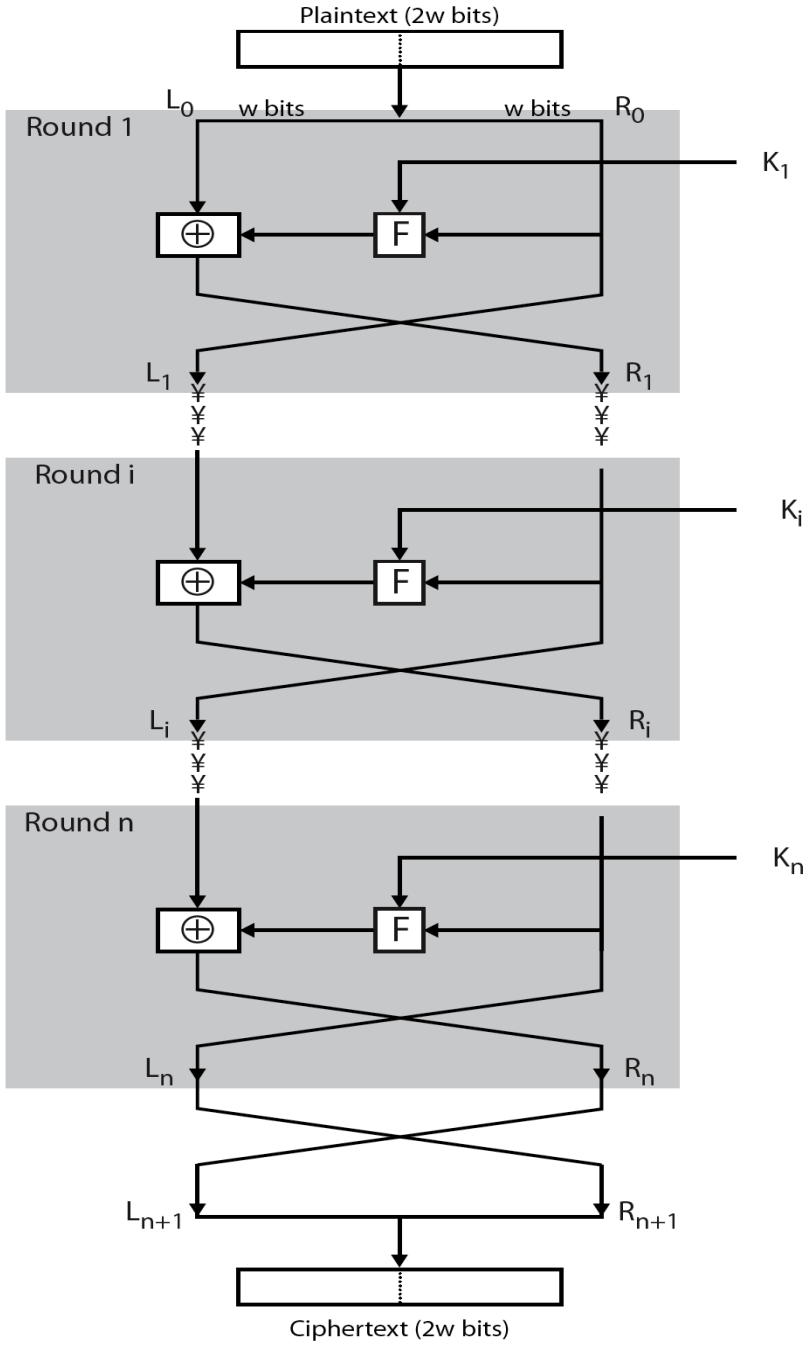
❖Gathering all the motivations described before, **Horst Feistel** in 1973 proposed an **approximation** to the ideal block cipher via **utilizing the product cipher**.

❖**Idea**:

- The execution of **two or more** simple ciphers in sequence such that the final output ciphertext is **cryptographically stronger** than executing **one**.
- To develop a block cipher with a **key length of $k$ bits** and a **block length of $n$ bits**, allowing a total of $2^k$ possible transformation, rather than $2^n!$ transformation as in the ideal block cipher.
- The execution is done using **substitution** (to achieve confusion) and **permutation** (to achieve diffusion) **alternately**.

# **Feistel Cipher** Structure

1. Input of the encryption algorithm: Plaintext block of length $2w$ bits and key, $K$.
2. The plaintext block is **divided into two halves**, $L_0$ and $R_0$.
3. The two halves pass through $n$ rounds of processing and then combine to become the ciphertext block.
4. Each round $i$ has as inputs $L_{i-1}$ and $R_{i-1}$ derived from previous rounds as well as a subkey $K_i$ derived from overall $K$.
5. In general each **subkey $K_i$ is different** from $K$ and from each other.

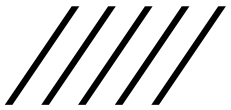


# **Feistel Cipher** Structure

❖**Properties** of Feistel cipher:

- ➢ All rounds have the **same** structure.
- ➢ A **substitution** is performed on the ***left half*** of the data by applying a **round function** $F$ to the ***right half*** of the data. The output is then **exclusive-OR** with the ***left half*** of the data.
- ➢ Each round function is the same except for the subkey, $k_i$ of that function.
- ➢ Following this substitution, a **permutation** is performed that consists of the interchange of the two halves of the data.

❖This structure is a particular form of **Substitution-Permutation (S-P) network** proposed by Shannon.



# Feistel Cipher – **Parameters & Design**

1. **Block size**:
   - Larger block size assures greater security, but reduce computational efficiency (speed).
   - Block size of 64 bits is traditionally nearly universal in block cipher design. But the new AES uses a 128-bit block size.
2. **Key size**:
   - Larger key size assures greater security to resist brute-force attack, but reduce computational efficiency (speed).
   - An adequate and common key size in deployment is 128 bits.
3. **Number of rounds**:
   - Multiple rounds performed to increase security strength.
   - A typical number of rounds is 16.

# Feistel Cipher – **Parameters & Design**

4. **Subkey generation algorithm**:
   - Greater complexity in this algorithm should lead to greater difficulty of cryptanalysis.
5. **Round function**:
   - Greater complexity generally means greater resistance to cryptanalysis
6. **Fast software encryption/decryption**:
   - Since encryption is embedded in applications or utility functions, the speed of execution of the algorithm becomes a concern.
7. **Ease of analysis**:
   - Algorithm should be easy to analyse for cryptanalytic vulnerabilities and develop a higher level of assurance as to its strength.

# DATA ENCRYPTION STANDARD (DES)

# Data Encryption Standard (DES) - **Introduction**

- DES is the most widely used encryption scheme adopted in 1977 by the National Bureau of Standard (now **National Institute of Standard and Technology (NIST)**).
- It encrypts **64-bit plaintext blocks** using a **56-bit key**, and outputs **64-bit ciphertext blocks** via a series of computations. The exact same computations and keys are used in decryption too.
- This block cipher enjoys widespread of application, but also creates **controversy** (we will discuss this in later slides) with respect to its **security**.

# Data Encryption Standard (DES) - **History**

- In the late 1973, a **LUCIFER** algorithm was designed by Feistel and Coppersmith, a project set up by IBM.
- The **LUCIFER** is a Feistel block cipher that operates in **64-bit plaintext blocks** using **128-bit key**. This algorithm was **used in cash-dispensing system**.
- Due to this promising result, IBM developed a **refined version of LUCIFER** with technical advice from National Security Agency (NSA).
  - More commercialized.
  - More resistant to cryptanalysis.
  - Reduced key-size (**56-bit key**) to fit on a single chip.

# Data Encryption Standard (DES) - **Controversy**

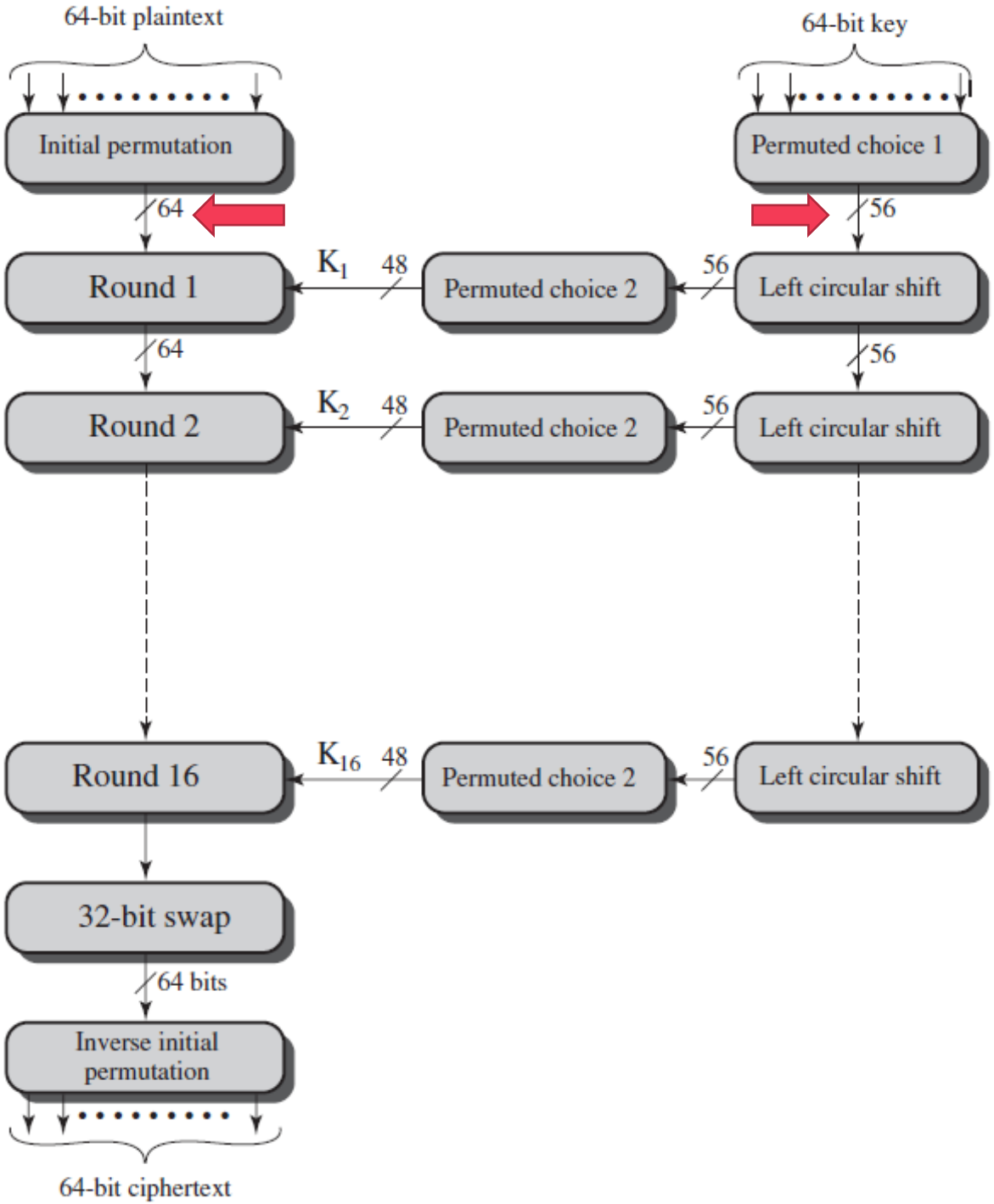
❖The **two main controversies** in DES as compared to LUCIFER are that:

1. **Choice of key size**:
   - LUCIFER uses **128-bit key** while DES uses only **56-bit key**. The possibility that shorter key length (with large reduction of **72-bit**) is vulnerable to **brute-force attack**.
2. **Internal structure DES**:
   - The design criteria of the internal structure of DES uses **S-Boxes that were classified**. There is no assurance about the security that is **weakness-free**.
   - Though, work in **differential cryptanalysis** indicates that the internal structure is very strong.

# DES – Encryption Algorithm (Overview)

- Two inputs to the encryption function:

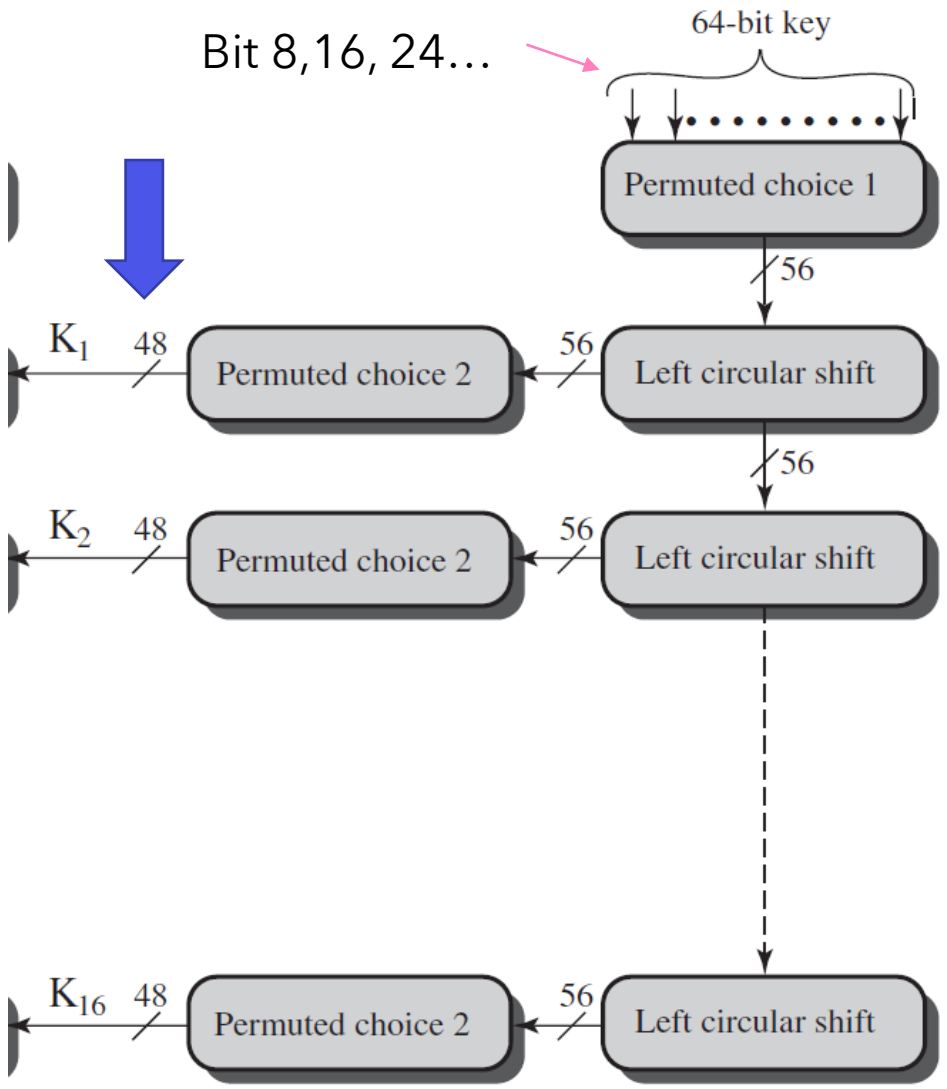
1. The 64-bit plaintext block. **Full 64-bit plaintext** block will be encrypted. (left-side of the chart).
2. The 64-bit key. **Only 56-bit key length** is used (right-side of the chart). **Every 8th-bit is discarded in Permuted choice 1**.



# DES – Encryption Algorithm (Key)

❖ **DES key**:

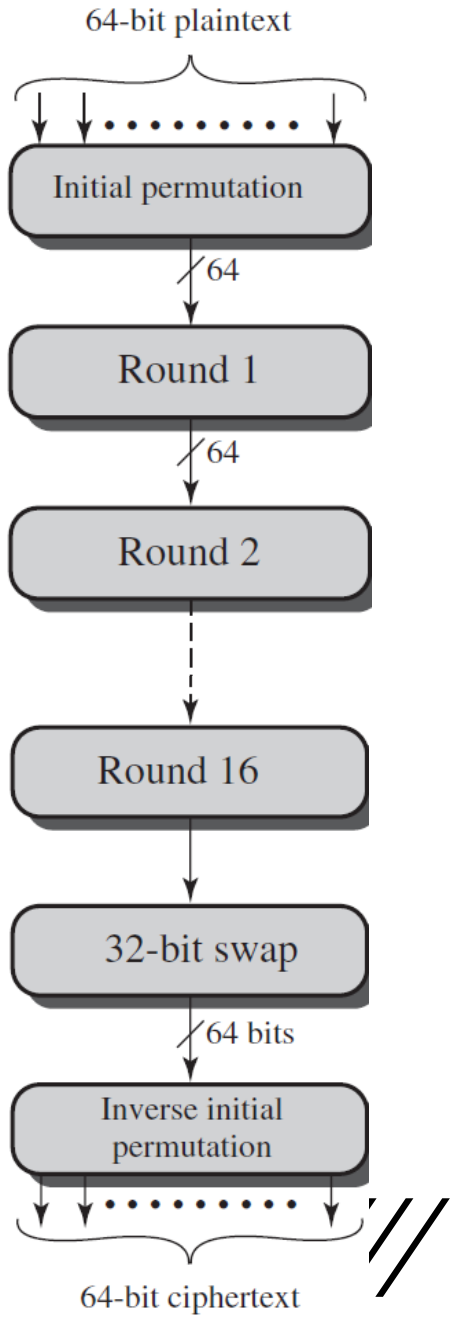
1. On input of a 64-bit key, **every $8^{th}$-bit is discarded** to produce 56-bit key.
2. The 56-bit key passes through **Permuted Choice 1**, **left circular shift**, followed by **Permuted Choice 2**.
3. The final output of Permuted Choice 2 produces **48-bit round key (subkey $K_i$)** that will be used for encryption in each round.



# DES – **Encryption Algorithm (Plaintext Block)**

❖ **Plaintext block**:

1. On input of a 64-bit plaintext block, it passes through an **Initial Permutation ($IP$)** to produce permuted input.
2. The permuted input will then runs through **16 rounds of identical function**, including **expansion**, **substitution**, and **permutation**.
3. The output of the left and right halves in the final round ($16^{th}$-round) are **swapped** to produce the **preoutput**.
4. The **preoutput** passes through **Inverse Initial Permutation ($IP^{-1}$)** to produce the 64-bit ciphertext.



# DES – **Single Round**

The figure shows the **complete single round** of DES algorithm.

We first learn how to generate the round key $K_i$, followed by the step-by-step operation in encryption functions.

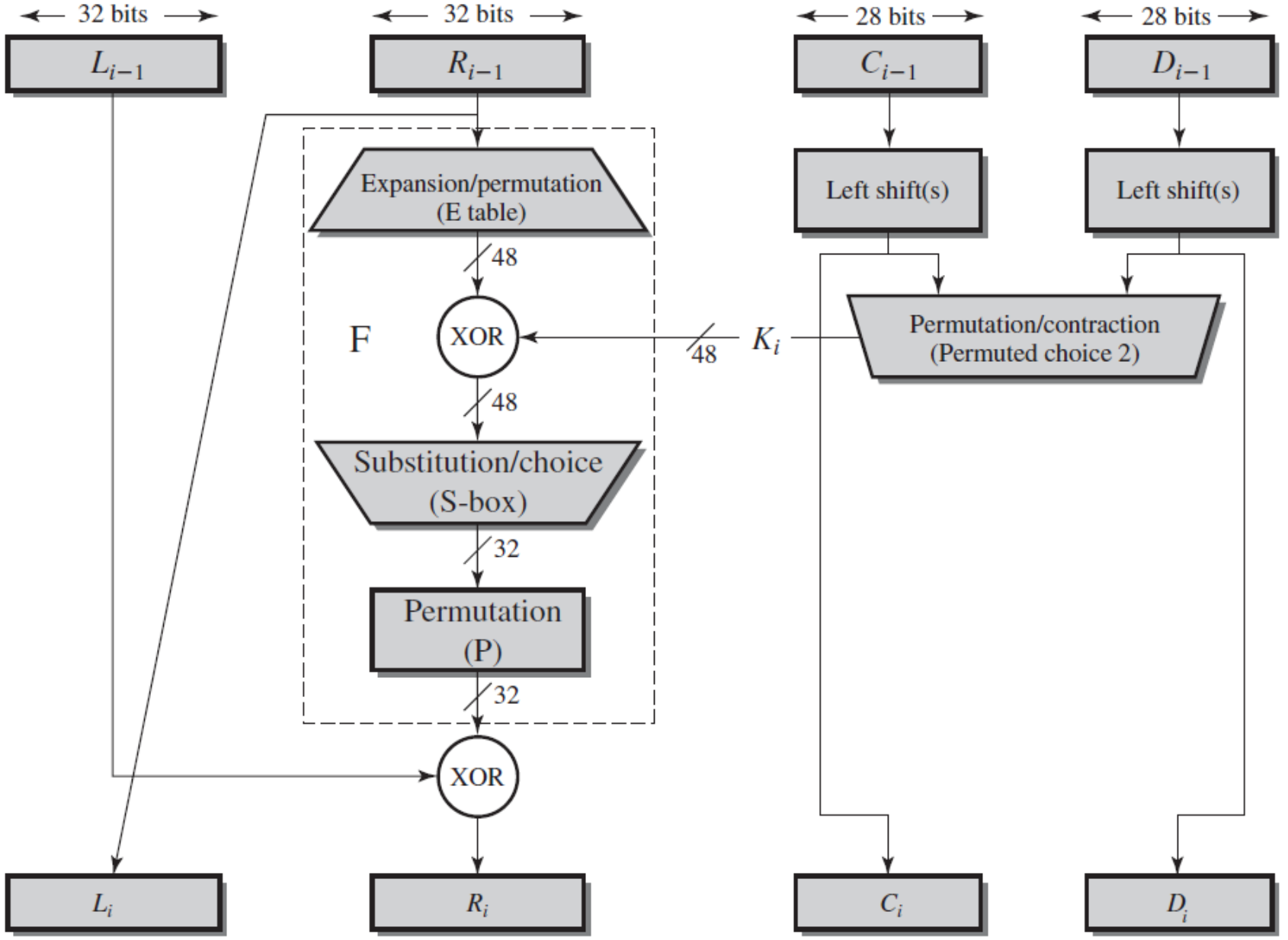
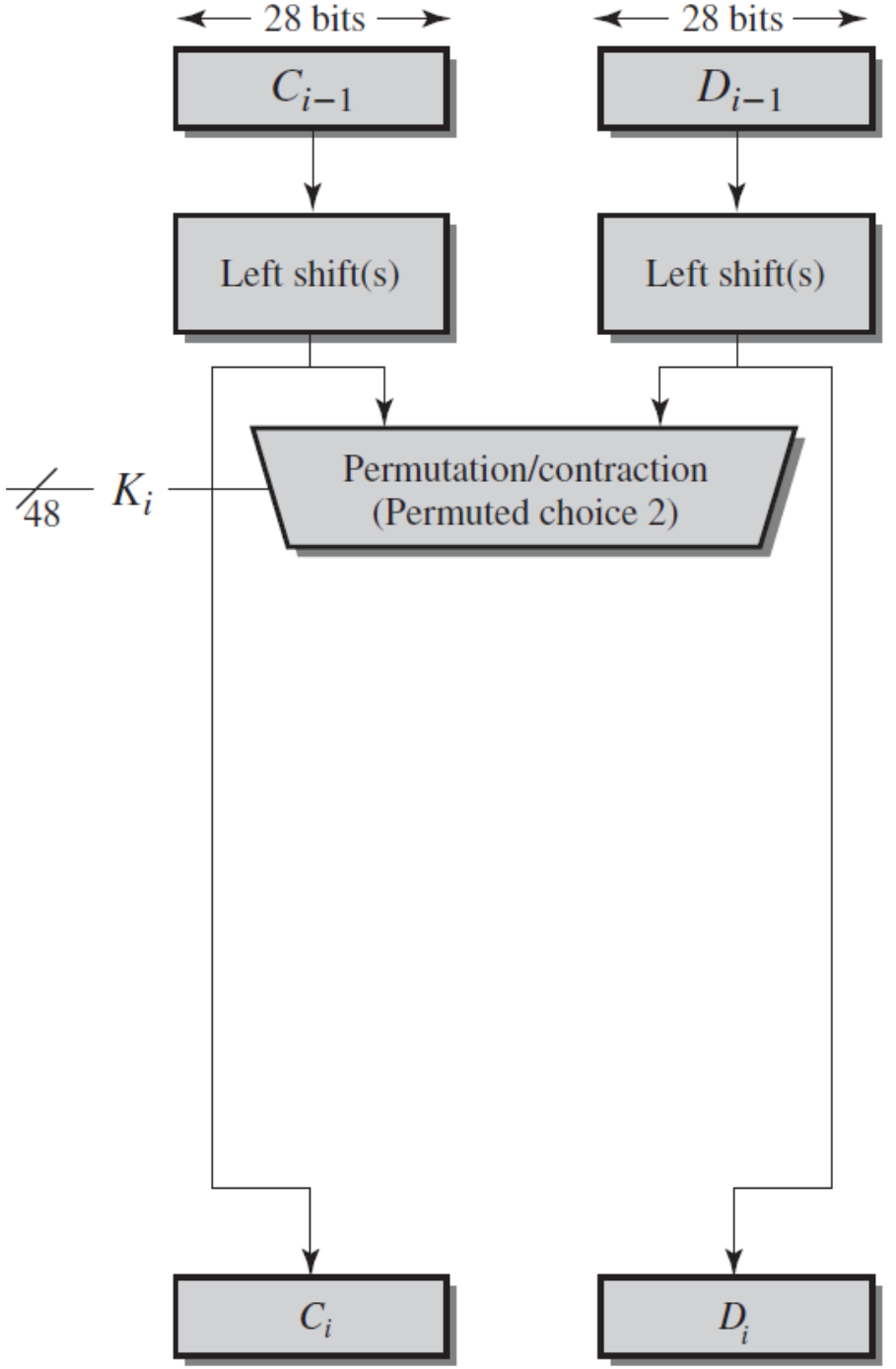



Figure 3.6 Single Round of DES Algorithm

# DATA ENCRYPTION STANDARD (DES)

## ROUND (SUBKEY) GENERATION

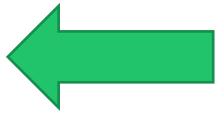


# DES – Key Generation (Input Key)

1. On input of a 64-bit key, **every 8th-bit is discarded** to produce 56-bit key.

**(a) Input Key**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 |
| 25 | 26 | 27 | 28 | 29 | 30 | 31 | 32 |
| 33 | 34 | 35 | 36 | 37 | 38 | 39 | 40 |
| 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 |
| 49 | 50 | 51 | 52 | 53 | 54 | 55 | 56 |
| 57 | 58 | 59 | 60 | 61 | 62 | 63 | 64 |



After arranging the key bits, **discard** the bit in **every 8 th** position.

# DES – Key Generation (Permuted Choice 1)

2. The 56-bit key passes through **Permuted Choice 1 (PC1)** according to the arrangement in the following table. Note that all the $8^{th}$-bit (8,16,24,32,40,48,56,64) are already discarded here.

**(b) Permuted Choice One (PC-1)**

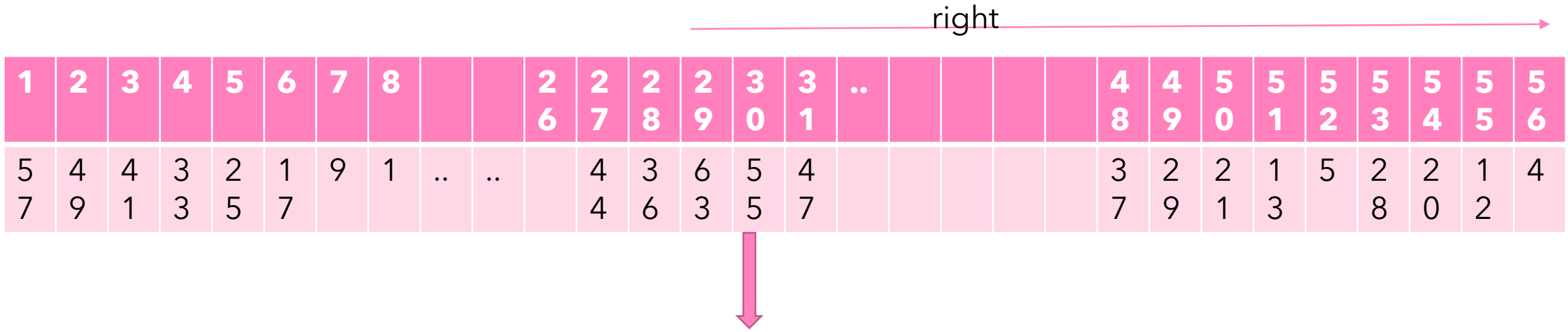
| | | | | | | |
|---|---|---|---|---|---|---|
| 57 | 49 | 41 | 33 | 25 | 17 | 9 |
| 1 | 58 | 50 | 42 | 34 | 26 | 18 |
| 10 | 2 | 59 | 51 | 43 | 35 | 27 |
| 19 | 11 | 3 | 60 | 52 | 44 | 36 |
| 63 | 55 | 47 | 39 | 31 | 23 | 15 |
| 7 | 62 | 54 | 46 | 38 | 30 | 22 |
| 14 | 6 | 61 | 53 | 45 | 37 | 29 |
| 21 | 13 | 5 | 28 | 20 | 12 | 4 |

LEFT

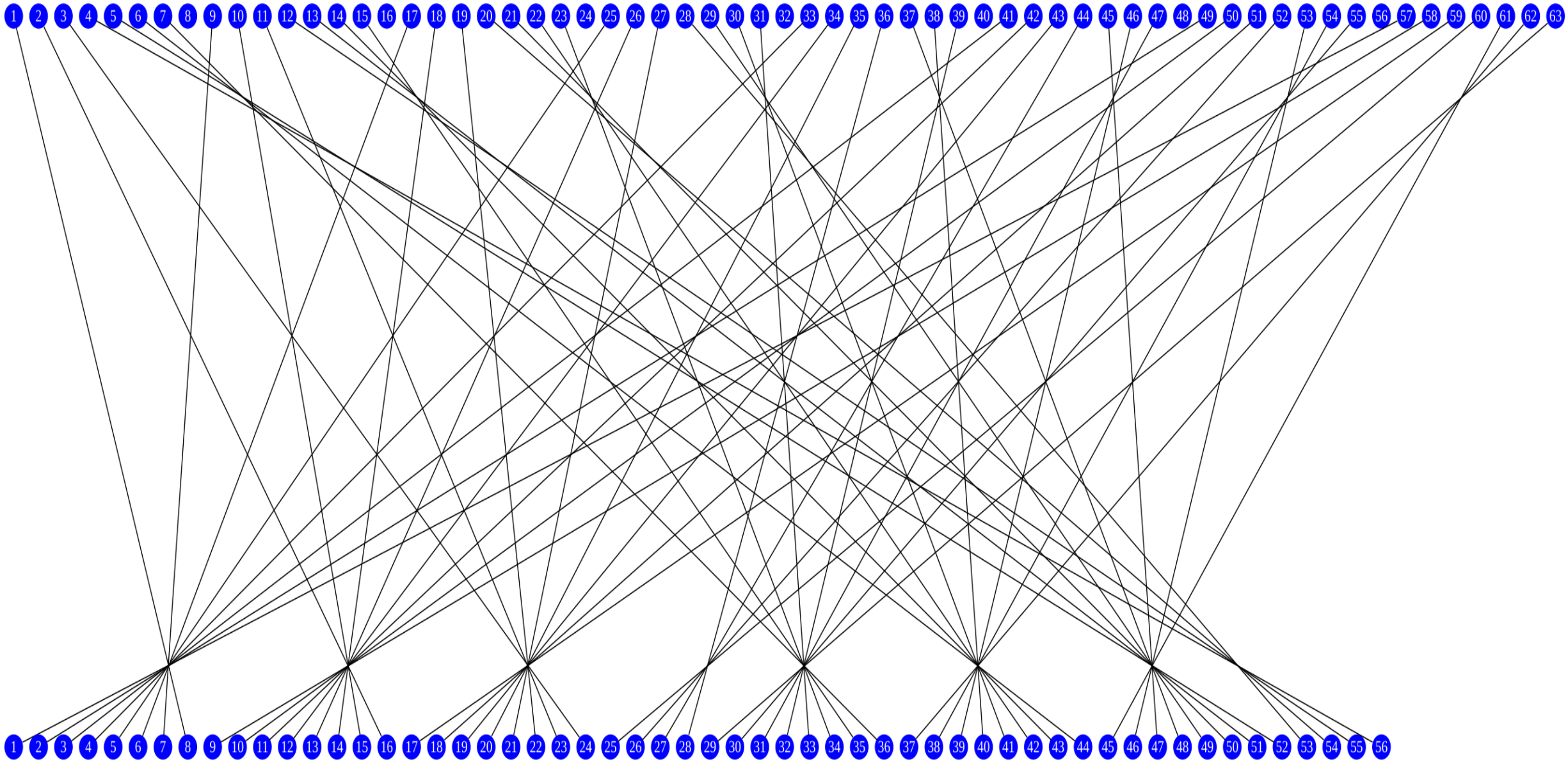
RIGHT

The output of the PC1 is immediately **parted into TWO halves** (left and right with 28 bits each)

# Permuted Choice 1



# Permutation Choice 1



# DES – **Key Generation (Left Circular Shift)**

3. Each half of left and right block undergoes left shifts according the current of subkey generation.
   - For **Round 1, 2, 9, and 16** subkeys generation, **ONE bit** is shifted to the left.
   - For the **other rounds** of subkey generation, **TWO bits** are shifted to the left.
   - 4 x 1 shift , 12 round x 2 = 24 + 4 = 28 bits x 2 = 56 bits

**(d) Schedule of Left Shifts**

| Round Number | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Bits Rotated | 1 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 1 | 2 | 2 | 2 | 2 | 2 | 2 | 1 |

# DES – Key Generation (Permuted Choice 2)

4. After the left circular shift, both left and right halves pass through the **Permuted Choice 2 (PC2)** for further compression into **48-bit keys**. This is the **Round key $K_i$** that will be used for **single round Feistel encryption**.

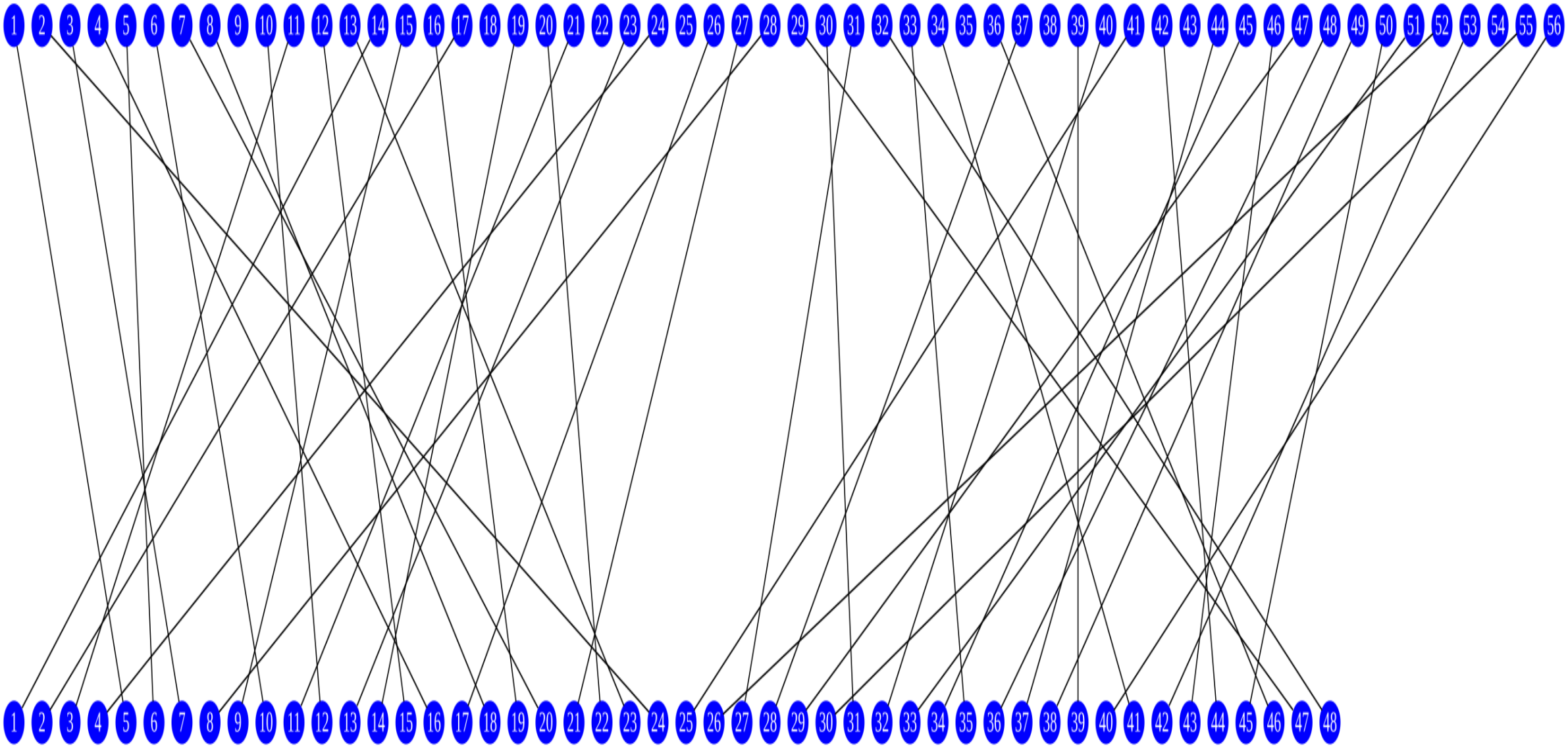
**(c) Permuted Choice Two (PC-2)**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 14 | 17 | 11 | 24 | 1 | 5 | 3 | 28 |
| 15 | 6 | 21 | 10 | 23 | 19 | 12 | 4 |
| 26 | 8 | 16 | 7 | 27 | 20 | 13 | 2 |
| 41 | 52 | 31 | 37 | 47 | 55 | 30 | 40 |
| 51 | 45 | 33 | 48 | 44 | 49 | 39 | 56 |
| 34 | 53 | 46 | 42 | 50 | 36 | 29 | 32 |

The output of the PC2 is a **48-bit key** instead of 56-bit.

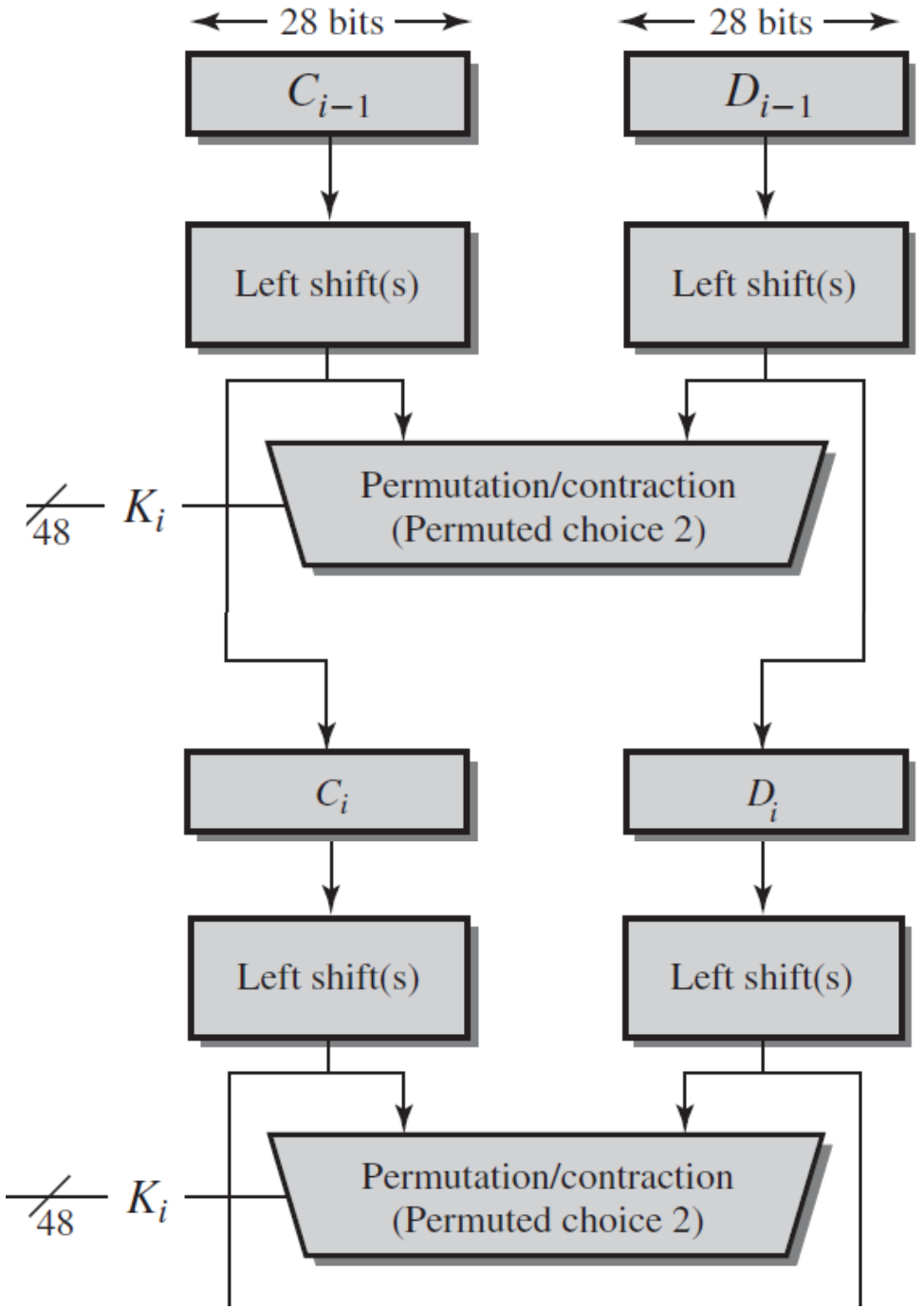
Ignore bit : 9, 18, 22, 25, 35, 38, 43, 54.

# Permuted Choice-2



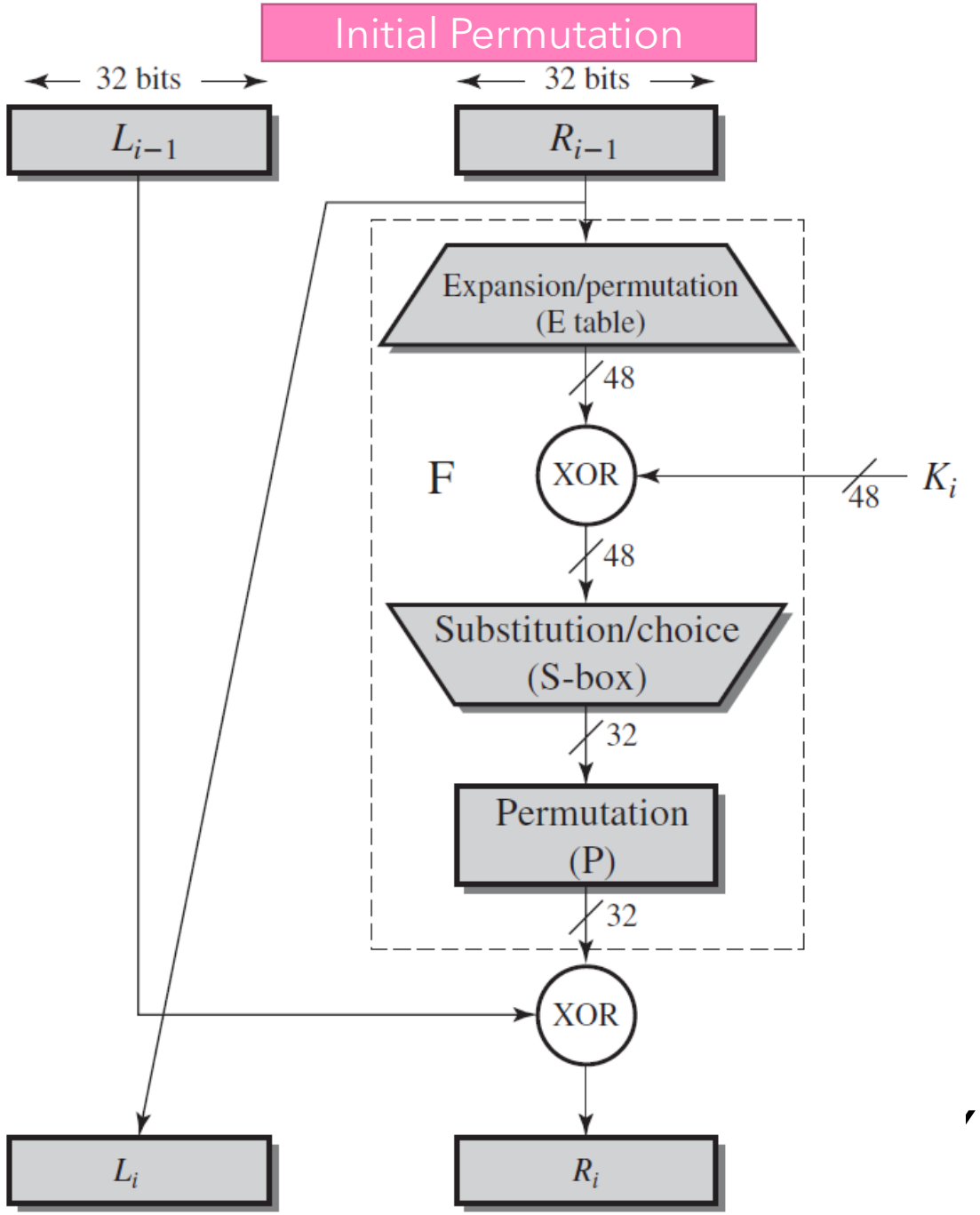
# DES – **Key Generation**

5. While passing the left and right halves to Permuted Choice 2, these left-shifted key blocks are also **repeating the identical operations** in **Step 3** to generator **subkey** in **NEXT round**.



# DATA ENCRYPTION STANDARD (DES)

## ENCRYPTION ALGORITHM

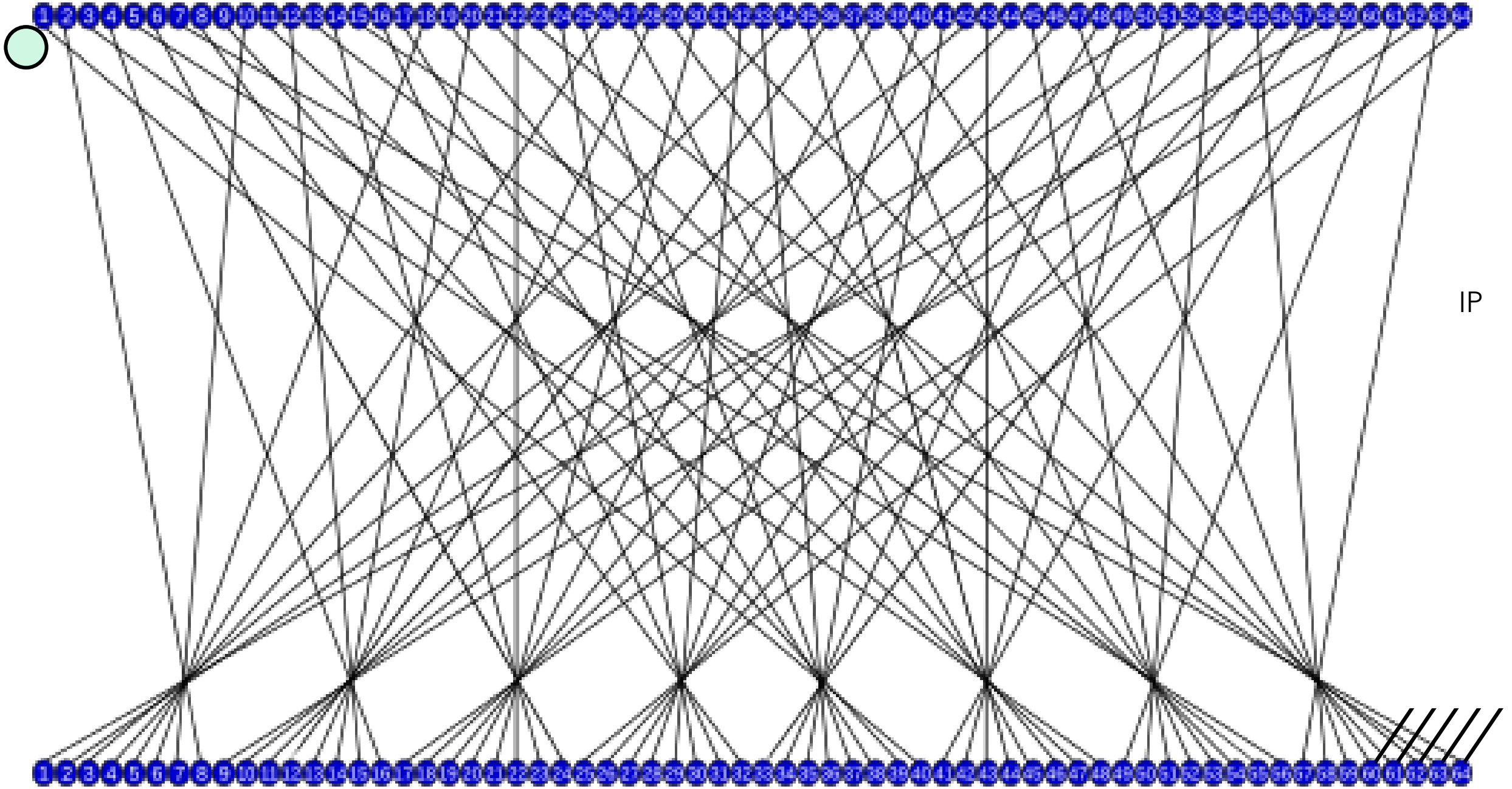


# DES – **Encryption Algorithm (Initial Permutation)**

1. On input of a 64-bit plaintext block, it is permuted using the **Initial Permutation ($IP$)** table as follows.
2. The **permuted plaintext block** is then split into two halves of **32-bit left block $L_i$** and **32-bit right block $R_i$**.

**(a) Initial Permutation (IP)**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 58 | 50 | 42 | 34 | 26 | 18 | 10 | 2 |
| 60 | 52 | 44 | 36 | 28 | 20 | 12 | 4 |
| 62 | 54 | 46 | 38 | 30 | 22 | 14 | 6 |
| 64 | 56 | 48 | 40 | 32 | 24 | 16 | 8 |
| 57 | 49 | 41 | 33 | 25 | 17 | 9 | 1 |
| 59 | 51 | 43 | 35 | 27 | 19 | 11 | 3 |
| 61 | 53 | 45 | 37 | 29 | 21 | 13 | 5 |
| 63 | 55 | 47 | 39 | 31 | 23 | 15 | 7 |


IP

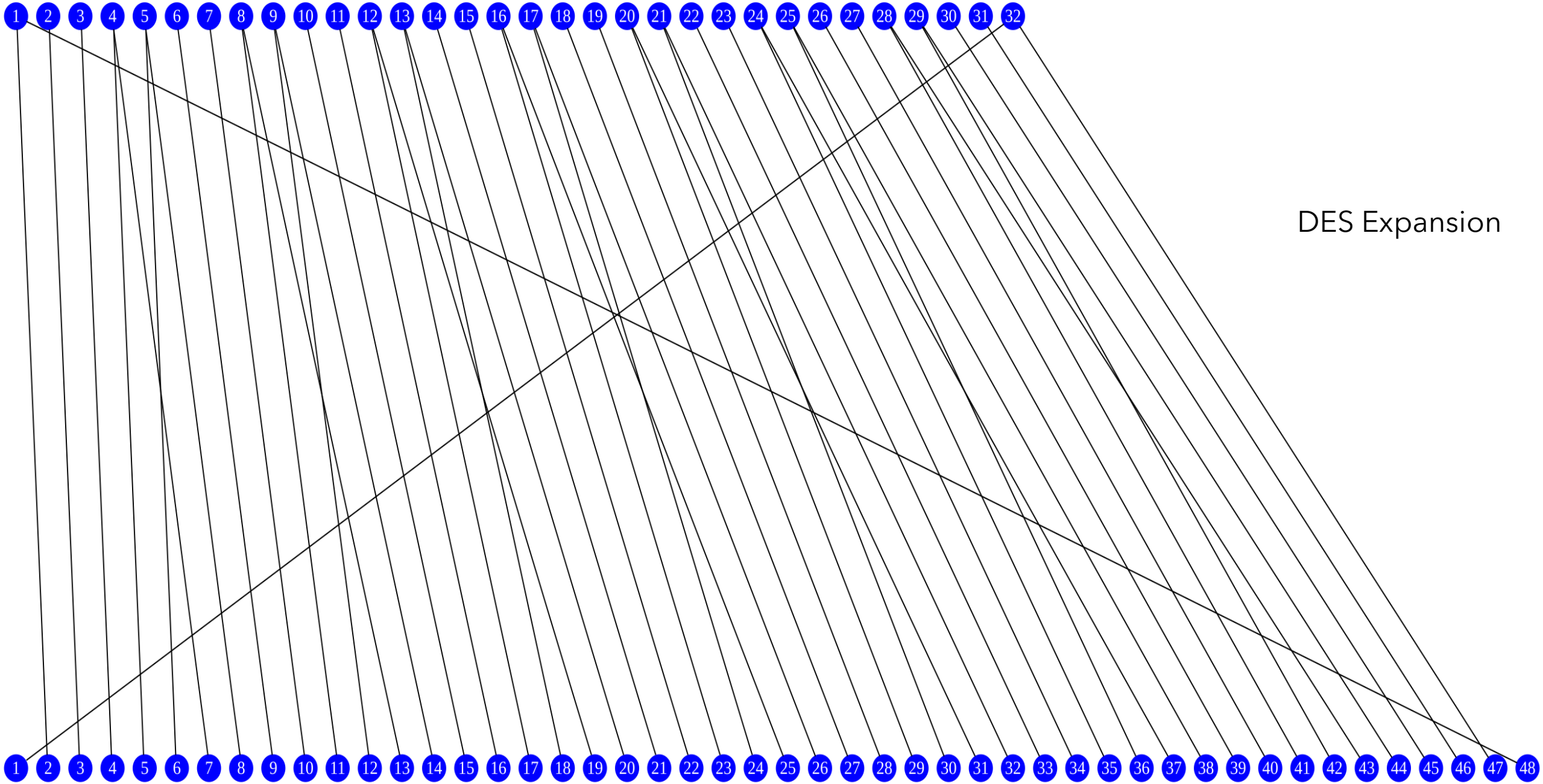
# DES – Encryption Algorithm (Expansion)

3. The **32-bit right block $R_i$** has **two purposes** here:

- It immediately becomes the **left ciphertext block $L_{i+1}$** for **NEXT** round. That is $L_{i+1} = R_i$
- It undergoes the expansion operation to form a **48-bit right block $E(R_i)$** using the **Expansion** permutation table as follows.
- 16 bits mapped to 16 bits
- eg. Bit 2 mapped to Bit 3. 1 bit map to 1 bit.
- 16 bits mapped to 32 bits . 1 bit map to 2 bit.
- eg. Bit 1,4,5. Bit 1 mapped to Bit 2, 48

**(c) Expansion Permutation (E)**

| | | | | | |
|---|---|---|---|---|---|
| 32 | 1 | 2 | 3 | 4 | 5 |
| 4 | 5 | 6 | 7 | 8 | 9 |
| 8 | 9 | 10 | 11 | 12 | 13 |
| 12 | 13 | 14 | 15 | 16 | 17 |
| 16 | 17 | 18 | 19 | 20 | 21 |
| 20 | 21 | 22 | 23 | 24 | 25 |
| 24 | 25 | 26 | 27 | 28 | 29 |
| 28 | 29 | 30 | 31 | 32 | 1 |




DES Expansion

# DES – **Encryption Algorithm (Exclusive-OR)**

4. The **expanded 48-bit right block $E(R_i)$** is then **XOR (exclusive-OR)** with the **subkey $K_i$** generated previously to produce a **48-bit block**. Mathmatically, $E(R_i) \oplus K_i$. Recall that exclusive-OR operations:

| Input Bit 1 | Input Bit 2 | Resultant Bit under $\oplus$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

# DES – **Encryption Algorithm (Substitution)**

5. The output of the **48-bit block** from **Step 4** is then split into **8 smaller 6-bit blocks** for the **substitution** operation using the DES **S-Boxes** to compress it into producing a **32-bit block output (8 smaller 4-bit blocks)**. These S-Boxes are used in such a way that:
   - The **first** and **last bit** in the corresponds to the **ROW** of the S-Boxes.
   - The **middle 4 bits** corresponds to the **COLUMN** of the S-Boxes.
   - By searching the **intersected row and column**, the integer found is the output of the S-Box substitution and is then **converted into it binary form**.

# S-Box

$S_1$

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 14 | 4 | 13 | 1 | 2 | 15 | 11 | 8 | 3 | 10 | 6 | 12 | 5 | 9 | 0 | 7 |
| 0 | 15 | 7 | 4 | 14 | 2 | 13 | 1 | 10 | 6 | 12 | 11 | 9 | 5 | 3 | 8 |
| 4 | 1 | 14 | 8 | 13 | 6 | 2 | 11 | 15 | 12 | 9 | 7 | 3 | 10 | 5 | 0 |
| 15 | 12 | 8 | 2 | 4 | 9 | 1 | 7 | 5 | 11 | 3 | 14 | 10 | 0 | 6 | 13 |

Input = 48 bits. 8 x 6 = 48

**8 S-Box. Each of them translate 6 bit =>. 4 bit.**
Output = 32 bits. 4 x 8 = 32

| Position | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| Value | 1 | 0 | 1 | 1 | 1 | 0 |

Row : 10 = 2 (Position 1 and 6)
Column : 0111 = 7 (position 2 to 5)
What is the value of [2,7] in S1 table ?
11 => 1011

101110 =➔ 1011

# S-Box: Exercise

$S_1$

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 14 | 4 | 13 | 1 | 2 | 15 | 11 | 8 | 3 | 10 | 6 | 12 | 5 | 9 | 0 | 7 |
| 0 | 15 | 7 | 4 | 14 | 2 | 13 | 1 | 10 | 6 | 12 | 11 | 9 | 5 | 3 | 8 |
| 4 | 1 | 14 | 8 | 13 | 6 | 2 | 11 | 15 | 12 | 9 | 7 | 3 | 10 | 5 | 0 |
| 15 | 12 | 8 | 2 | 4 | 9 | 1 | 7 | 5 | 11 | 3 | 14 | 10 | 0 | 6 | 13 |

Input = 48 bits. 8 x 6 = 48

**8 S-Box. Each of them translate 6 bit =>. 4 bit.**

Output = 32 bits. 4 x 8 = 32

| Position | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| Value | 1 | 0 | 1 | 1 | 1 | 0 |

Row : 10 = 2 (Position 1 and 6) red
Column : 0111 = 7 (position 2 to 5)
What is the value of [2,7] in S1 table ?
11 => 1011
Thus,
101110 =➔ 1011

Question:
If input = 011011
What is the output for S1?
⇒ row1, column 13
⇒ 5
⇒ 011011 => 0101

# DES – **Encryption Algorithm (Substitution)**

$S_1$

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 14 | 4 | 13 | 1 | 2 | 15 | 11 | 8 | 3 | 10 | 6 | 12 | 5 | 9 | 0 | 7 |
| 0 | 15 | 7 | 4 | 14 | 2 | 13 | 1 | 10 | 6 | 12 | 11 | 9 | 5 | 3 | 8 |
| 4 | 1 | 14 | 8 | 13 | 6 | 2 | 11 | 15 | 12 | 9 | 7 | 3 | 10 | 5 | 0 |
| 15 | 12 | 8 | 2 | 4 | 9 | 1 | 7 | 5 | 11 | 3 | 14 | 10 | 0 | 6 | 13 |

$S_2$

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 15 | 1 | 8 | 14 | 6 | 11 | 3 | 4 | 9 | 7 | 2 | 13 | 12 | 0 | 5 | 10 |
| 3 | 13 | 4 | 7 | 15 | 2 | 8 | 14 | 12 | 0 | 1 | 10 | 6 | 9 | 11 | 5 |
| 0 | 14 | 7 | 11 | 10 | 4 | 13 | 1 | 5 | 8 | 12 | 6 | 9 | 3 | 2 | 15 |
| 13 | 8 | 10 | 1 | 3 | 15 | 4 | 2 | 11 | 6 | 7 | 12 | 0 | 5 | 14 | 9 |

$S_3$

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 10 | 0 | 9 | 14 | 6 | 3 | 15 | 5 | 1 | 13 | 12 | 7 | 11 | 4 | 2 | 8 |
| 13 | 7 | 0 | 9 | 3 | 4 | 6 | 10 | 2 | 8 | 5 | 14 | 12 | 11 | 15 | 1 |
| 13 | 6 | 4 | 9 | 8 | 15 | 3 | 0 | 11 | 1 | 2 | 12 | 5 | 10 | 14 | 7 |
| 1 | 10 | 13 | 0 | 6 | 9 | 8 | 7 | 4 | 15 | 14 | 3 | 11 | 5 | 2 | 12 |

$S_4$

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 7 | 13 | 14 | 3 | 0 | 6 | 9 | 10 | 1 | 2 | 8 | 5 | 11 | 12 | 4 | 15 |
| 13 | 8 | 11 | 5 | 6 | 15 | 0 | 3 | 4 | 7 | 2 | 12 | 1 | 10 | 14 | 9 |
| 10 | 6 | 9 | 0 | 12 | 11 | 7 | 13 | 15 | 1 | 3 | 14 | 5 | 2 | 8 | 4 |
| 3 | 15 | 0 | 6 | 10 | 1 | 13 | 8 | 9 | 4 | 5 | 11 | 12 | 7 | 2 | 14 |

$S_5$

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 2 | 12 | 4 | 1 | 7 | 10 | 11 | 6 | 8 | 5 | 3 | 15 | 13 | 0 | 14 | 9 |
| 14 | 11 | 2 | 12 | 4 | 7 | 13 | 1 | 5 | 0 | 15 | 10 | 3 | 9 | 8 | 6 |
| 4 | 2 | 1 | 11 | 10 | 13 | 7 | 8 | 15 | 9 | 12 | 5 | 6 | 3 | 0 | 14 |
| 11 | 8 | 12 | 7 | 1 | 14 | 2 | 13 | 6 | 15 | 0 | 9 | 10 | 4 | 5 | 3 |

$S_6$

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 12 | 1 | 10 | 15 | 9 | 2 | 6 | 8 | 0 | 13 | 3 | 4 | 14 | 7 | 5 | 11 |
| 10 | 15 | 4 | 2 | 7 | 12 | 9 | 5 | 6 | 1 | 13 | 14 | 0 | 11 | 3 | 8 |
| 9 | 14 | 15 | 5 | 2 | 8 | 12 | 3 | 7 | 0 | 4 | 10 | 1 | 13 | 11 | 6 |
| 4 | 3 | 2 | 12 | 9 | 5 | 15 | 10 | 11 | 14 | 1 | 7 | 6 | 0 | 8 | 13 |

$S_7$

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 4 | 11 | 2 | 14 | 15 | 0 | 8 | 13 | 3 | 12 | 9 | 7 | 5 | 10 | 6 | 1 |
| 13 | 0 | 11 | 7 | 4 | 9 | 1 | 10 | 14 | 3 | 5 | 12 | 2 | 15 | 8 | 6 |
| 1 | 4 | 11 | 13 | 12 | 3 | 7 | 14 | 10 | 15 | 6 | 8 | 0 | 5 | 9 | 2 |
| 6 | 11 | 13 | 8 | 1 | 4 | 10 | 7 | 9 | 5 | 0 | 15 | 14 | 2 | 3 | 12 |

$S_8$

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 13 | 2 | 8 | 4 | 6 | 15 | 11 | 1 | 10 | 9 | 3 | 14 | 5 | 0 | 12 | 7 |
| 1 | 15 | 13 | 8 | 10 | 3 | 7 | 4 | 12 | 5 | 6 | 11 | 0 | 14 | 9 | 2 |
| 7 | 11 | 4 | 1 | 9 | 12 | 14 | 2 | 0 | 6 | 10 | 13 | 15 | 3 | 5 | 8 |
| 2 | 1 | 14 | 7 | 4 | 10 | 8 | 13 | 15 | 12 | 9 | 0 | 3 | 5 | 6 | 11 |

# DES – **Encryption Algorithm (Substitution)**

- **Example**: Assuming we are now looking at the 3rd 6-bit block from the encryption algorithm. Find the substitution of the block **101101**.

- **Solution**: The first and last bit **11** corresponds to **4th row**, and **0110** corresponds to **7th column**. By searching the integer from S-Box 3, we have integer $8 = 0111$ in binary. Hence, **0111 is the resultant 4-bit block from substitution**.

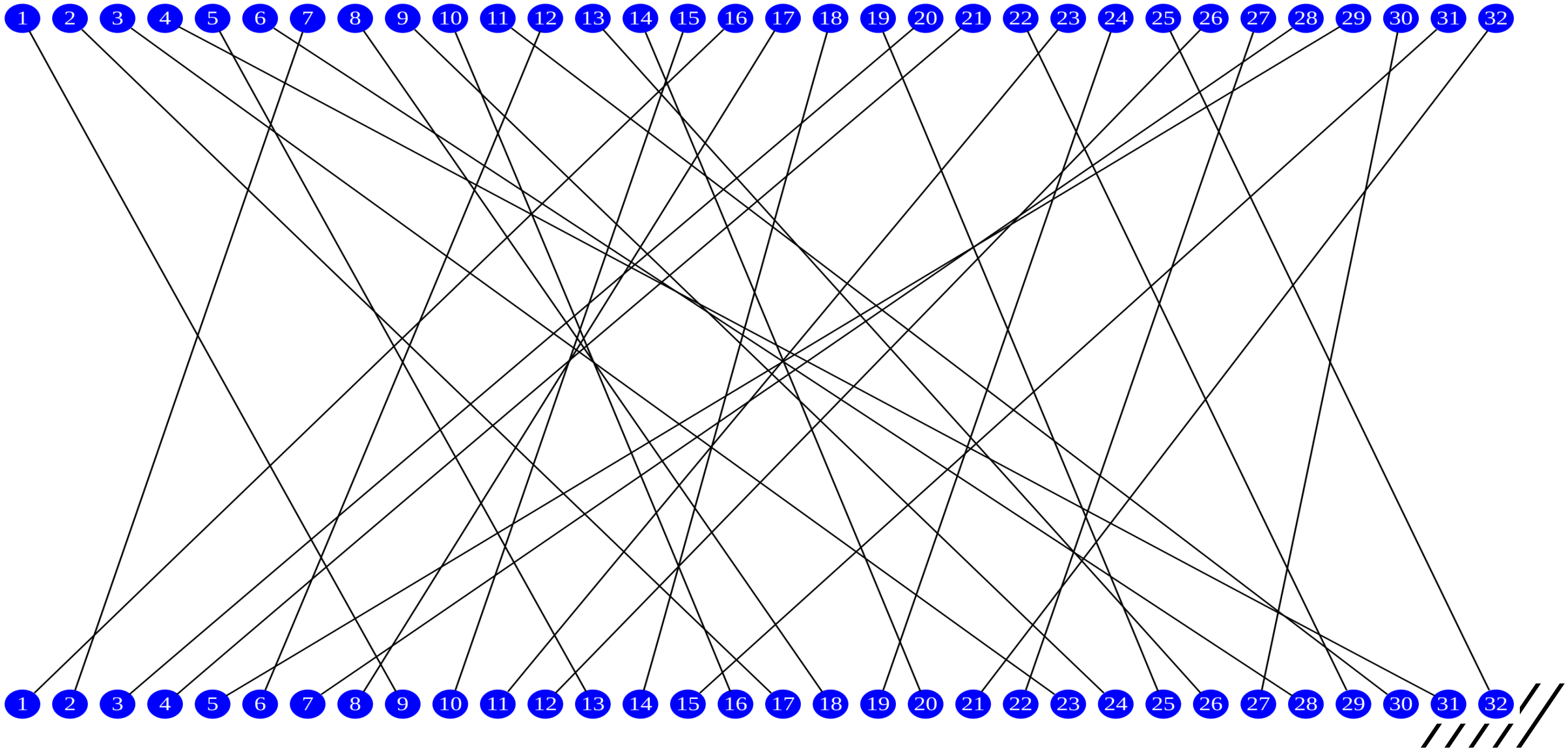
# DES – **Encryption Algorithm (Permutation)**

6. The output compressed **32-bit block** is next undergoes the permutation using the P-table as follows.

**(d) Permutation Function (P)**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 16 | 7 | 20 | 21 | 29 | 12 | 28 | 17 |
| 1 | 15 | 23 | 26 | 5 | 18 | 31 | 10 |
| 2 | 8 | 24 | 14 | 32 | 27 | 3 | 9 |
| 19 | 13 | 30 | 6 | 22 | 11 | 4 | 25 |

# Permutation



# DES – **Encryption Algorithm (Exclusive-OR)**

7. The permuted **32-bit block** is then XOR with the **32-bit left block $L_i$**, producing the final output of **32-bit right ciphertext block $R_{i+1}$** for the **NEXT** round. ***This complete Round 1 encryption operations***.
   - Combining both the left block and right block, we generally express them is mathematics as

$$L_{i+1} = R_i$$
$$R_{i+1} = L_i \oplus F(R_i, K_i)$$

   where $F(R_i, K_i)$ is the function described in Step 3 until Step 7.

8. **Step 3 until Step 7** are **repeated** for another **15 rounds**.

# DES – **Encryption Algorithm (Inverse Initial Permutation)**

9. At the end of Round 16, the left and right ciphertext blocks $L_{16}$ and $R_{16}$ are **swapped** and will undergo the last **inverse initial permutation ($IP^{-1}$)** using the following table to produce the **final output DES ciphertext block**.

**(b) Inverse Initial Permutation ($IP^{-1}$)**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 40 | 8 | 48 | 16 | 56 | 24 | 64 | 32 |
| 39 | 7 | 47 | 15 | 55 | 23 | 63 | 31 |
| 38 | 6 | 46 | 14 | 54 | 22 | 62 | 30 |
| 37 | 5 | 45 | 13 | 53 | 21 | 61 | 29 |
| 36 | 4 | 44 | 12 | 52 | 20 | 60 | 28 |
| 35 | 3 | 43 | 11 | 51 | 19 | 59 | 27 |
| 34 | 2 | 42 | 10 | 50 | 18 | 58 | 26 |
| 33 | 1 | 41 | 9 | 49 | 17 | 57 | 25 |

# DES – **Decryption Algorithm**

❖For the **decryption of DES**, the operations are identical to encryption, except the **order** subkeys used is **reversed**, from **Round 16 subkey** and move backwardly to **Round 1 subkey**.

❖One can refer to the mathematical equations below to decrypt the DES ciphertext.

$$R_{i-1} = L_i$$

$$L_{i-1} = R_i \oplus F(L_i, K_i)$$

# DES – **The Avalanche Effect**

- A factor that makes DES a preferred symmetric encryption is its strong **Avalanche effect**, a property that is desirably by any encryption algorithm.

**Avalanche effect**:

A ***change of one bit*** of plaintext input or key bit should result in changing ***approximately half*** of the output ciphertext bits.

- It is very difficult to demonstrate an example here. Please refer to the **Stallings textbook**, **page 86–87** for an example using 128-bit plaintext block.

# DATA ENCRYPTION STANDARD (DES)

## VARIANTS

# Multiple DES - **Motivation**

- The need of replacement for DES in critical since the current available technology and the possibility of parallel processing in computing ability, resulting higher feasibility for a **brute-force attack** on DES.
  - ➢ An attacker just need to exhaustively search the $2^{56}$ **possible keys in DES**.

- **Potential solutions**:

1. Utilize **stronger symmetric encryption** as an alternative, such as designing a new **Advanced Encryption Standard (AES)** for a better security.
2. Use **multiple instances DES** with **multiple keys**, since it does not require additional investment in both software and hardware.

# DES Variant – **Double DES (2DES)**

- This is the simplest form of multiple DES encryptions that has **two encryption stages** and **two keys**.
- Given a plaintext $\mathbf{P}$ and two encryption keys $K_1$ and $K_2$, ciphertext $\mathbf{C}$ is generated as:

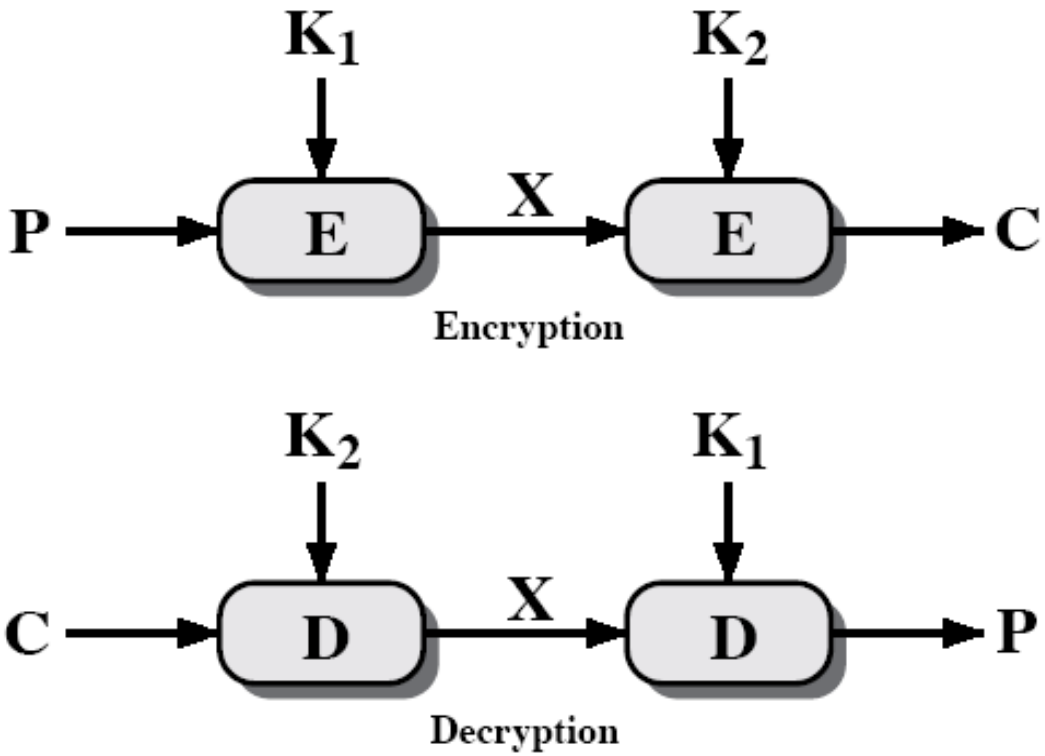
$$\mathbf{C} = \mathbf{Enc}\big(K_2, \mathbf{Enc}(\mathbf{P}, K_1)\big)$$

- Decryption requires that the key be applied in reverse order:

$$\mathbf{P} = \mathbf{Dec}\big(K_1, \mathbf{Dec}(\mathbf{C}, K_2)\big)$$

- For 2DES, this involves a key length of $56 \times 2 = 112$**-bits** and seems to result in a dramatic increase in cryptographic strength.
  - However, using a **known-plaintext attack** called a **meet-in-the-middle attack** proves that 2DES improves this vulnerability slightly (to $2^{57}$ tests), but not tremendously (to $2^{112}$).

# DES Variant – **Double DES (2DES)**




$$C = Enc(K_2, Enc(P, K_1))$$

$$P = Dec(K_1, Dec(C, K_2))$$

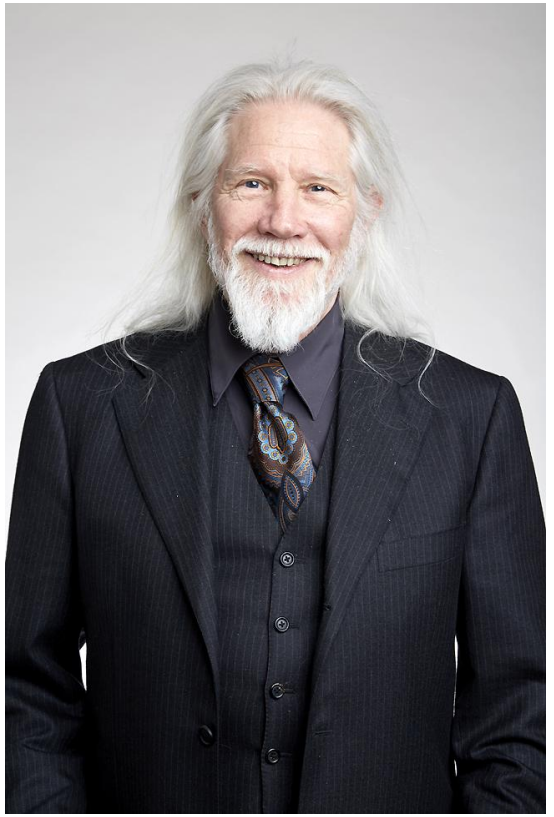
# 2DES – Meet-in-the-Middle Attack

- ✓An example of a **Known-Plaintext Attack (KPA)**, **Meet-in-the-Middle** attack was firstly described by Whitfield Diffie in 1977.
- ✓This attempts to find by **trial-and-error** a value $\boldsymbol{X}$ in the '**middle**' of the 2DES encryption.
- ✓This algorithm is based on the observation that if we have

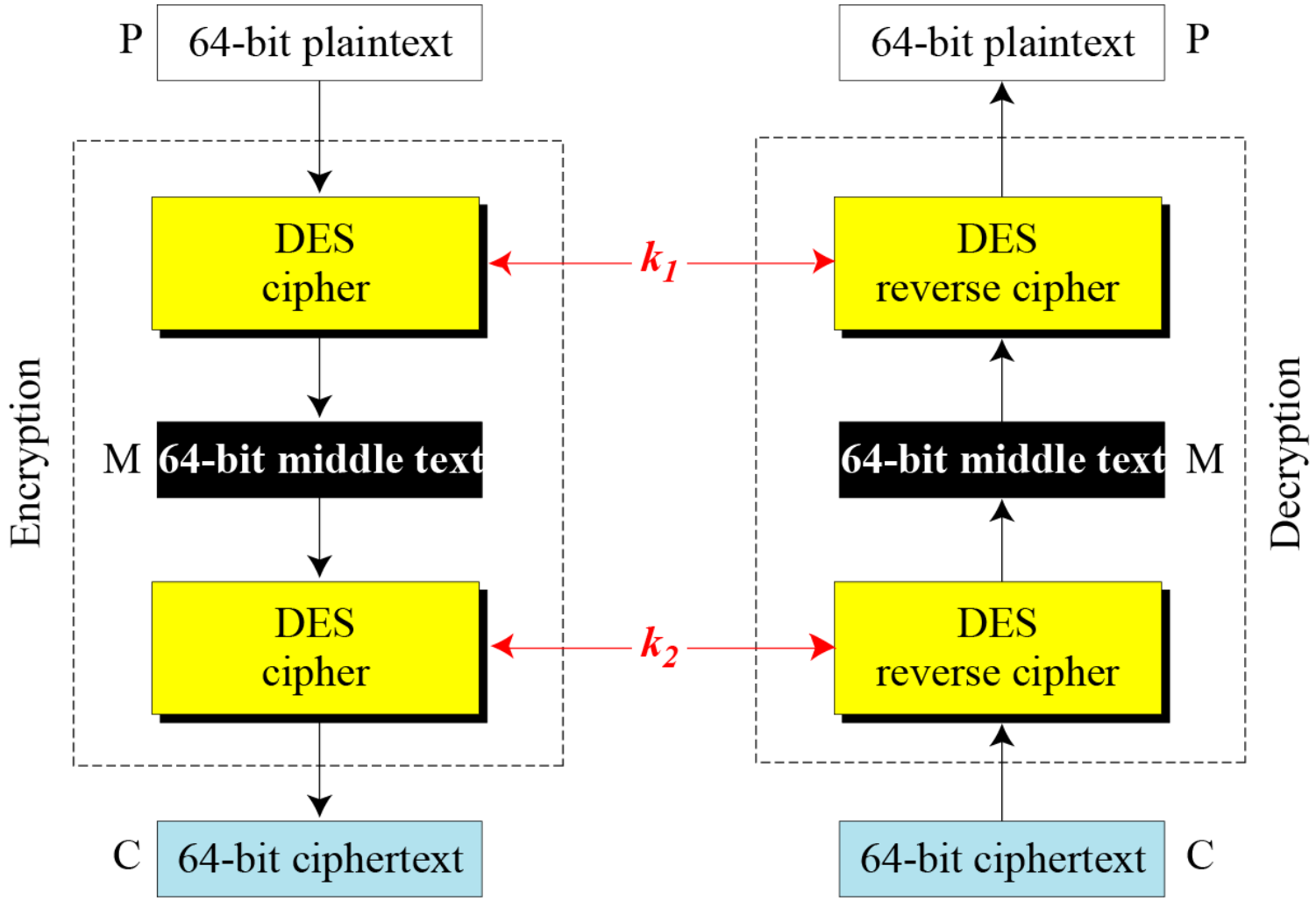
$$\mathbf{C} = \mathrm{Enc}\big(K_2, \mathrm{Enc}(K_1, \mathbf{P})\big)$$

then

$$\mathrm{Dec}(K_2, \mathbf{C}) = \mathrm{Enc}(K_1, \mathbf{P}) = \boldsymbol{X}\ (\mathbf{Eq.\ 1})$$



# 2DES – Meet-in-the-Middle Attack



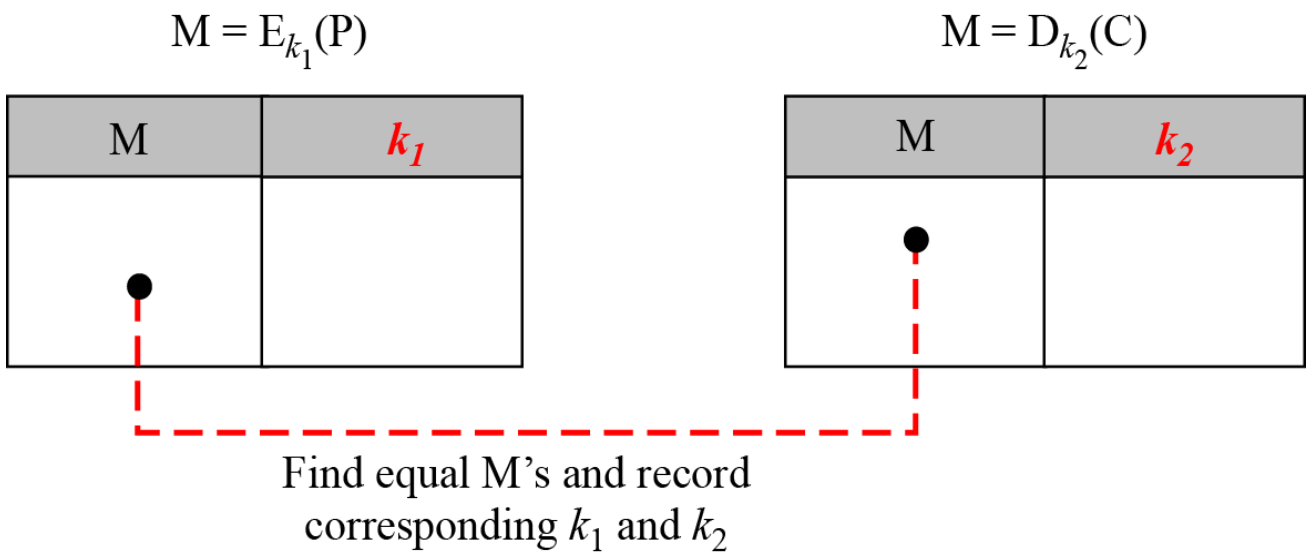
# 2DES – Meet-in-the-Middle Attack

➢ How does Meet-in-the-Middle attack work?

1. Assume that an attacker has intercepted a previous pair of plaintext-ciphertext $(\mathbf{P}, \mathbf{C})$ (**known-plaintext attack**).
2. Based on the **first relationship**, the attacker encrypts $\mathbf{P}$ using all possible values $(2^{56})$ of $k_1$ and records all values obtained for $\mathbf{M}$.
3. Based on the **second relationship**, the attacker decrypts $\mathbf{C}$ using all possible values $(2^{56})$ of $k_2$ and records all values obtained for $\mathbf{M}$.
4. The attacker creates two tables sorted by $\mathbf{M}$ values and compare these two tables.
5. If there is **only one match** of $\mathbf{M}$ found, then the attacker has **found the two keys** $k_1$ and $k_2$. Otherwise, go to the next step.

# 2DES – Meet-in-the-Middle Attack

6. The attacker takes another intercepted plaintext-ciphertext pair and uses each of the candidate key pairs to see if it is possible to get the ciphertext from the plaintext.
7. If more than one candidate key pair is found, the above Step 6 is **repeated** until a unique pair of $(k_1, k_2)$ that produces the identical $\mathbf{M}$ is found.



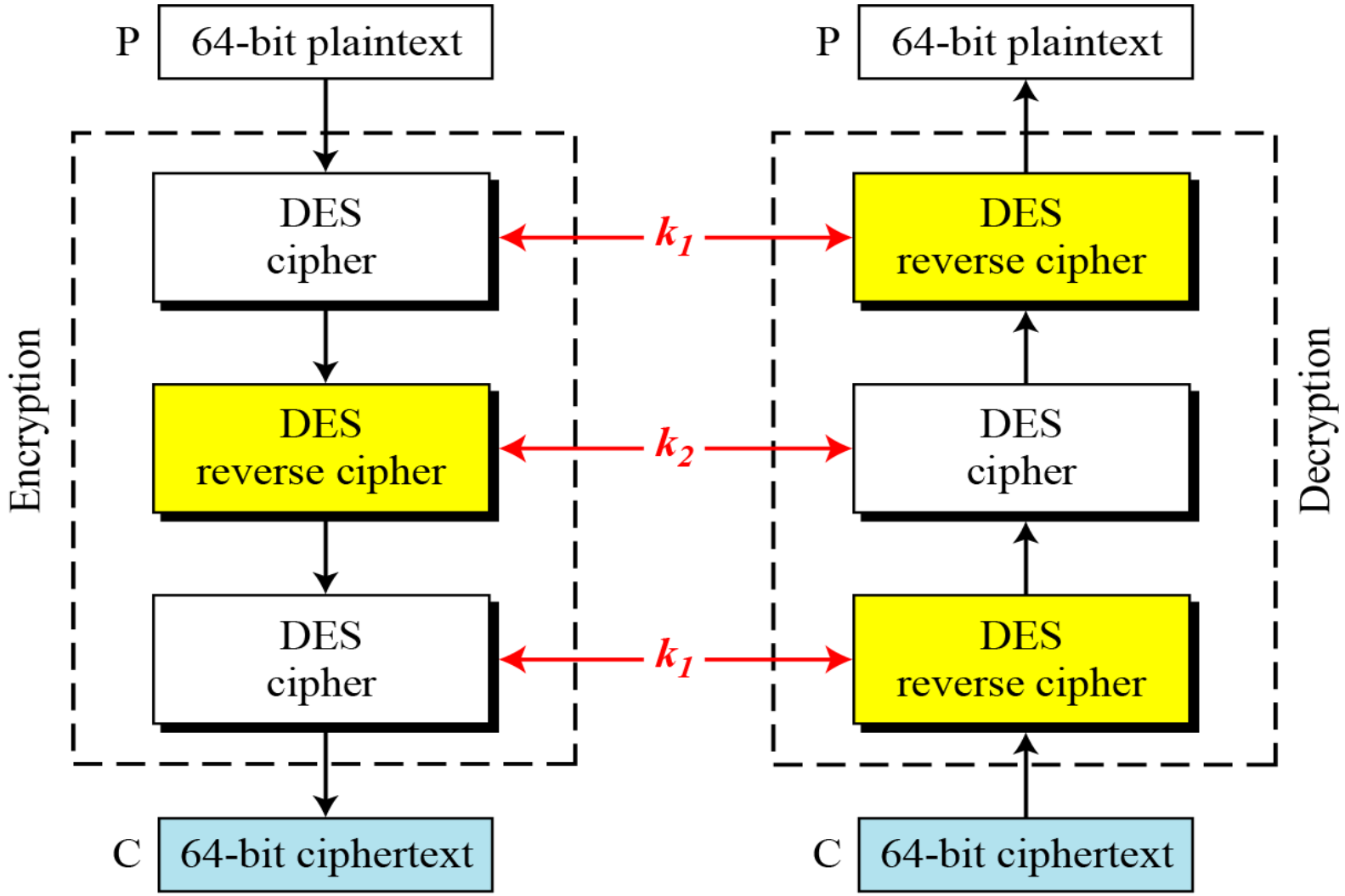
# DES Variant – **Triple DES (3DES)**

- Another variant of DES proposed to improve its security, **Triple DES (3DES)** uses three stages of DES for encryption and decryption.
- There are **two version of 3DES** in use today:
  1. 3DES with **2 keys**.
  2. 3DES with **3 keys**.

# Triple DES (3DES) with 2 Keys

- There are only two keys of $k_1$ and $k_2$ involved, with the **first** and **third** stages use $\boldsymbol{k_1}$, and the **second** stage uses $\boldsymbol{k_2}$.
- To make 3DES compatible with single DES, the 2nd stage uses **reverse cipher technique** in its algorithm.
  - The plaintext $\mathbf{P}$ is **encrypted using $\boldsymbol{k_1}$**, **decrypt using $\boldsymbol{k_2}$**, and **encrypt using $\boldsymbol{k_1}$** again to produce ciphertext $\mathbf{C}$.
  - The ciphertext $\mathbf{C}$ is **decrypted using $\boldsymbol{k_1}$**, **encrypt using $\boldsymbol{k_2}$**, and **decrypt using $\boldsymbol{k_1}$** again to recover plaintext $\mathbf{P}$.
- Although 3DES with 2 keys is also vulnerable to known-plaintext attack, it is nevertheless much stronger than 2DES.

# Triple DES (3DES) with 2 Keys



# **Triple DES (3DES)** with **3 Keys**

- In this version of 3DES, three keys $k_1$, $k_2$ and $k_3$ are used., with each round is operated using its corresponding key.
- To make the 3DES with 3 keys compatible with single DES, the same **reverse cipher technique** as in 3DES with 2 keys are implemented in its algorithm.
  - The plaintext $\mathbf{P}$ is **encrypted using $k_1$**, **decrypt using $k_2$**, and **encrypt using $k_3$** to produce ciphertext $\mathbf{C}$.
  - The ciphertext $\mathbf{C}$ is **decrypted using $k_3$**, **encrypt using $k_2$**, and **decrypt using $k_1$** to recover plaintext $\mathbf{P}$.
- 3DES with 3 keys is used in many applications, one of the famous application is the **PGP (Pretty Good Privacy)**.

# Main References for Chapter 4


1. Stallings, W. "***Cryptography and Network Security: Principles and Practices***", Fifth Edition, Pearson Prentice Hall, 2011.
2. Forouzan, B.A. "***Introduction to Cryptography and Network Security***", Third Edition, McGraw-Hill, 2008.

# Summary

- Key generation
- Encryption Fiestel Function.
  - => Expansion. 32 bit => 48 bits.

# Grouping

- Opt 1: Individual component 1st, Then join a group 2-3 person from other component.
- Opt 2: Group straight away. Do 2-3 components

~ THE END ☺ ~